CIE-1 STUDY GUIDE · UNIT 1 OF 3 · CS51

Unit 1
Process & Agile

SOFTWARE ENGINEERING & MODELLING

What software engineering is, how software is built step by step (process models), and how modern teams build it in short iterations (agile, XP, Scrum).

IN THIS UNIT

Chapter 1 · Introduction to Software Engineering

Chapter 2 · Software Processes

Chapter 3 · Agile Software Development

SYLLABUS

Introduction: Professional software development, Software engineering ethics.

Software processes: Software process models, Process activities, Coping with change, Process improvement.

Agile Software Development: Agile methods, Agile development techniques, Agile project management.

Built from **your lecture slides**, **Sommerville (10th ed)** and **Pressman**.

Course coordinators
Mr. Pradeep Kumar D · Dr Shilpa S Chaudhari

Start here: how to use this guide

This guide covers **exactly the CIE-1 portion** of Software Engineering & Modelling (CS51): all of Unit 1, all of Unit 2, and Unit 3 up to **architectural views**. It assumes you know nothing yet. Every point from your lecture slides is here, explained in plain English first, then in the textbook wording you should use in answers.

Unit	Syllabus topics (CIE-1)	Where in this guide
Unit 1	**Introduction:** professional software development, software engineering ethics	Chapter 1 · Introduction
Unit 1	**Software processes:** process models, process activities, coping with change, process improvement	Chapter 2 · Software Processes
Unit 1	**Agile software development:** agile methods, agile development techniques, agile project management	Chapter 3 · Agile Development
Unit 2	**Requirements engineering:** functional & non-functional requirements, RE processes, elicitation, specification, validation, change	Chapter 4 · Requirements Engineering
Unit 2	**Design concepts:** design within the context of SE, design concepts, the design model	Chapter 5 · Design Concepts
Unit 3	**Architectural design:** architectural design decisions, architectural views (patterns onwards are after CIE-1)	Chapter 6 · Architectural Design

How each chapter works

1. **Opener page**: what the chapter covers, how long it takes and what you will be able to do.
2. **Sections**: each idea is explained simply, then given in exam wording, with an example and a diagram where one helps.
3. **Chapter in one page**: a dense revision sheet. Re-read it the night before the exam.
4. **Likely exam questions**: each one says what to write and which diagram to draw.

What the coloured boxes mean

DEFINITION — LEARN THIS LINE
A definition to learn **word for word**. Write it as the first line of your answer.

★ KEY POINT
A fact you must remember. These are the things examiners look for.

IN SIMPLE WORDS
The idea in everyday language. Read this first if a topic feels confusing.

EXAMPLE
A worked example, usually from the textbook case studies.

✎ EXAM TIP
How to write the answer: structure, diagram, keywords.

⚠ WATCH OUT
A common confusion or mistake that loses marks.

MEMORY TRICK
A trick for remembering a list.

? QUICK CHECK
Test yourself. Cover the answer and try it first.

Highlighted phrases are the exact keywords to use in your answers.

Priority badges

★★★ MUST KNOW

Asked often: definitions, models, diagrams, comparisons. Study these first and practise the diagrams.

★★ IMPORTANT

Commonly asked as short questions or parts of long answers. Understand them well.

★ READ ONCE

Background or lower-priority material. Read it once so nothing surprises you.

Study plans

If you have a week

- **Day 1**: Ch 1 + start Ch 2
- **Day 2**: finish Ch 2 and draw every model
- **Day 3**: Ch 3
- **Day 4**: Ch 4
- **Day 5**: Ch 5 + Ch 6
- **Day 6**: all question banks
- **Day 7**: one-page recaps + diagrams

If you have 3 days

- **Day 1**: Ch 1–3, focusing on ★★★ MUST KNOW
- **Day 2**: Ch 4–6, focusing on ★★★ MUST KNOW
- **Day 3**: question banks, then recaps; redraw every diagram once

If it's the night before

- Read all six **one-page recaps**
- Redraw the must-know diagrams from memory
- Skim each question bank and say the answer outline out loud
- Sleep; recall is better when you're rested

✎ GOLDEN RULE FOR SE ANSWERS

For anything worth more than 2 marks: **definition → diagram → explanation in points → example → advantages/disadvantages (or comparison)**. A neat, labelled diagram earns marks even when your text is short.

⚠ NOT IN CIE-1, SO SKIP THESE FOR NOW

Unit 3 from **architectural patterns** onwards (MVC, layered, repository, client–server, pipe-and-filter, application architectures), object-oriented design using UML, design patterns and implementation issues. Also all of **Unit 4** (testing, evolution) and **Unit 5** (project management, planning, quality). They come after CIE-1.

Contents · Unit 1

UNIT 1 **Software Process & Agile Development** 4

CH 1 **Introduction to Software Engineering** 5

1.1 Why software engineering matters 5
1.2 Professional software development 6
1.3 Software products and product specification 7
1.4 Essential attributes of good software 8
1.5 What is software engineering? 9
1.6 General issues that affect software 11
1.7 Software engineering diversity and application types 11
1.8 Internet (web-based) software engineering 13
1.9 Software engineering ethics 13
1.10 Case studies used throughout the course 15
Chapter 1 in one page 19
Likely exam questions 20

CH 2 **Software Processes** 21

2.1 The software process 21
2.2 Software process models (Sommerville) 23
2.3 More process models (from Textbook 2, Pressman) 27
2.4 Process activities 31
2.5 Coping with change 35
2.6 Process improvement 38
2.7 Comparing all the process models 40
2.8 Comparative analysis on the case studies (lab) 41
Chapter 2 in one page 43
Likely exam questions 44

CH 3 **Agile Software Development** 45

3.1 Rapid and agile development 45
3.2 Agile methods 47
3.3 Agile techniques: Extreme Programming 49
3.4 User stories for requirements 51
3.5 Refactoring 52
3.6 Test-first development 53
3.7 Pair programming 55
3.8 Agile project management and Scrum 55
3.9 Scaling agile & agile vs plan-driven in practice 58
Chapter 3 in one page 61
Likely exam questions 61

EXAM **Exam-Day Playbook** 63

CIE-1 · UNIT 1

Unit 1: Software Process & Agile Development

1

What software engineering is, how software is built step by step (process models), and how modern teams build it in short iterations (agile, XP, Scrum).

SYLLABUS, WORD FOR WORD

Introduction: Professional software development, Software engineering ethics.

Software processes: Software process models, Process activities, Coping with change, Process improvement.

Agile Software Development: Agile methods, Agile development techniques, Agile project management.

CHAPTERS IN THIS UNIT

CHAPTER 1 — **Introduction to Software Engineering**
Professional software development · Software engineering ethics · Case studies — page 5

CHAPTER 2 — **Software Processes**
Software process models · Process activities · Coping with change · Process improvement — page 21

CHAPTER 3 — **Agile Software Development**
Agile methods · Extreme Programming & agile techniques · Agile project management (Scrum) · Scaling agile — page 45

COURSE OUTCOME

CO1: Explain professional software development practices, process models, ethical aspects, and apply agile principles and project management techniques in software engineering.

LAB COMPONENT

Comparative analysis of software process models on the real-time case study application.

CIE-1 SCOPE

Whole unit is in CIE-1.

UNIT 1 · CHAPTER 1

Introduction to Software Engineering

Professional software development · Software engineering ethics · Case studies

STUDY TIME
About 2.5 hours

SOURCES
Your slides (Ch 1, 57 slides) + Sommerville Ch 1

EXAM WEIGHT
High: definitions, attributes of good software, application types and the code of ethics are classic 2–10M questions

AFTER THIS CHAPTER YOU CAN

- Define software and software engineering
- Compare generic and customized products
- Explain the four essential attributes of good software
- Name the four software process activities
- Differentiate SE, computer science and system engineering
- Explain the issues that affect software and the 8 application types
- Describe how the web changed software engineering
- Explain professional responsibility and the 8 ACM/IEEE principles

1.1 Why software engineering matters ★★ IMPORTANT

Banks, power grids, phones, cars, hospitals, films and games all run on software. Because so much depends on it, building software is a serious professional job, and that job is **software engineering**.

- The **economies of ALL developed nations are dependent on software**. Infrastructure, utilities, manufacturing and the financial system are all computer-controlled.
- **More and more systems are software controlled**: most electrical products now contain a computer and controlling software.
- Software engineering is concerned with theories, methods and tools for professional software development.
- **Expenditure on software** is a significant fraction of **GNP** (Gross National Product, the total value a country produces) in all developed countries.

Software costs

- Software costs often **dominate computer system costs**. The software on a PC often costs more than the hardware.
- Software **costs more to maintain than it does to develop**. For systems with a long life, maintenance costs may be *several times* development costs.
- So software engineering is concerned with cost-effective software development.



Fig 1.1 The cost "iceberg": most cost comes after delivery (not to scale). Drawn for this guide

Why do software projects fail? ★★★ MUST KNOW

Many software projects still go wrong. Sommerville traces most "software failures" to two factors:

1. **Increasing system complexity**: new software engineering techniques let us build larger, more complex systems, and then the demands change. Systems have to be built and delivered more quickly, larger and even more complex systems are required, and systems need new capabilities that were previously thought impossible.
2. **Failure to use software engineering methods**: it is fairly easy to write programs without software engineering methods. Many companies have *drifted* into software development as their products and services evolved, and they do not use these methods in their everyday work. Consequently, their software is often **more expensive and less reliable** than it should be.

1.2 Professional software development ★★★ MUST KNOW

Lots of people write programs for themselves, but most software development is a **professional activity**: software built for business purposes, for inclusion in other devices, or as software products.

★ KEY POINT

Professional software is intended for use by someone other than its developer, is usually built by **teams** rather than individuals, and is **maintained and changed throughout its life**. Software engineering supports this kind of development (not individual programming) with techniques for specification, design and evolution.

What is "software"? ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software: computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

So software is **not just programs**. It also includes the **documentation, libraries, support websites and configuration data** that make them useful: separate programs, configuration files, **system documentation** (the system's structure), **user documentation** (how to use it) and product websites. A personal program needs none of this, because nobody else will use or change it.

Frequently asked questions about software engineering ★★★ MUST KNOW

This table (Sommerville Fig 1.1, slides 7–8) summarises the subject in ten answers. Each row is a ready-made 2-mark answer.

Question	Answer
What is software?	Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.
What are the attributes of good software?	Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.
What is software engineering?	An engineering discipline that is concerned with all aspects of software production (textbook adds: "from initial conception to operation and maintenance").
What are the fundamental software engineering activities?	Software specification, software development, software validation and software evolution.
Software engineering vs computer science?	Computer science focuses on theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
Software engineering vs system engineering?	System engineering is concerned with all aspects of computer-based systems development, including hardware, software and process engineering. Software engineering is part of this more general process.
What are the key challenges facing software engineering?	Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.
What are the costs of software engineering?	Roughly 60% of software costs are development costs and 40% are testing costs. For custom software, evolution costs often exceed development costs.
What are the best software engineering techniques and methods?	All projects must be professionally managed and developed, but different techniques suit different types of system. For example, games should always be developed using a series of prototypes, whereas safety-critical control systems require a complete and analyzable specification. You cannot say that one method is better than another.
What differences has the web made to software engineering?	It has led to the availability of software services and the possibility of developing highly distributed, service-based systems. Web-based development has led to important advances in programming languages and software reuse. (The textbook adds that the Internet created an "app" industry for mobile devices.)

1.3 Software products and product specification ★★★ MUST KNOW

Software engineers are concerned with developing **software products**, which is software that can be sold to a customer. There are two kinds:

	Generic products	Customized (bespoke) products
What	Stand-alone systems that are marketed and sold to **any customer** who wishes to buy them.	Software that is **commissioned by a specific customer** to meet their own needs; a contractor builds it especially for them.

	Generic products	Customized (bespoke) products
Examples	PC software such as graphics programs and project management tools; CAD software; software for specific markets such as appointments systems for dentists; mobile apps.	Embedded control systems, air traffic control software, traffic monitoring systems; systems supporting one particular business process.
Product specification	The specification of what the software should do is owned by the software developer, and decisions on software change are made by the developer.	The specification is owned by the customer for the software, and they make decisions on the software changes that are required.

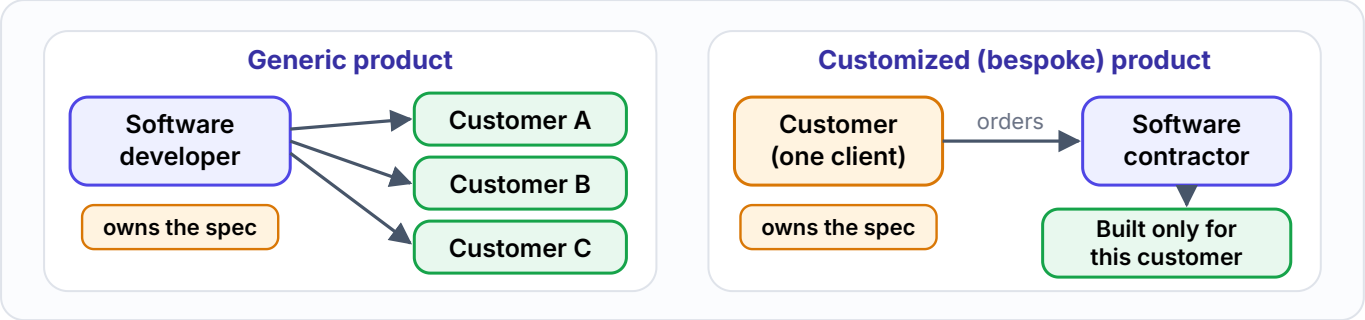


Fig 1.2 Generic vs customized products: the key difference is who owns the specification, and so who decides on changes. Drawn for this guide

Generic software is like a ready-made shirt: the maker decides the design, and can rethink it if development runs into problems. Customized software is like a tailor-made suit: the customer supplies the measurements (the specification) and the developers must work to them. The line is becoming **blurred**: many systems start from a generic product adapted for one customer, e.g. **ERP (Enterprise Resource Planning)** systems from SAP and Oracle, configured with a company's business rules, processes and reports.

1.4 Essential attributes of good software ★★★ MUST KNOW

Good software must first deliver the **required functionality and performance**. Beyond that, quality covers how it *behaves while running* and how its code and documents are *structured*. These are its **quality** or **non-functional attributes**, e.g. response time or how understandable the code is. The ones that matter most depend on the application: an aircraft control system must be **safe**, a game **responsive**, a telephone switch **reliable**. In general there are four:

Product characteristic	Description
Maintainability	Software should be written in such a way that it can **evolve to meet the changing needs of customers**. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics, including **reliability, security and safety**. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make **wasteful use of system resources** such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.

Product characteristic	Description
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be **understandable, usable and compatible** with other systems that they use.

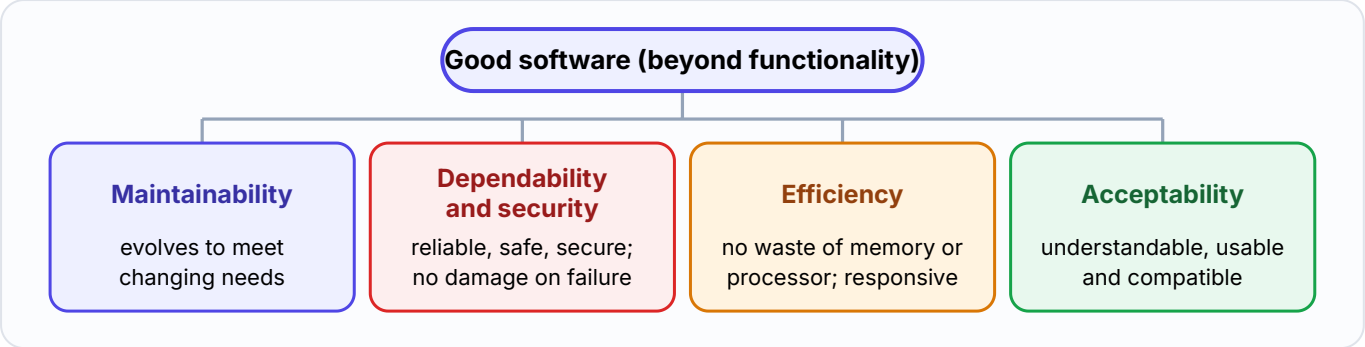


Fig 1.3 The four essential attributes of good software. Based on Sommerville Fig 1.2

MEMORY TRICK

M-D-E-A: "My Dog Eats Apples": **M**aintainability, **D**ependability & security, **E**fficiency, **A**cceptability. Example: for the **insulin pump**, dependability (safety, reliability) matters most; for **Mentcare**, security of patient records and acceptability to busy clinicians.

⚠ WATCH OUT

The FAQ says "maintainable, dependable and usable", but the expected list has **four names**: Maintainability, Dependability and security, Efficiency, Acceptability. Dependability covers reliability, security and safety.

1.5 What is software engineering? ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Software engineering: an engineering discipline that is concerned with all aspects of software production, from the early stages of system specification through to maintaining the system after it has gone into use.

Key phrase 1: Engineering discipline

Using **appropriate theories and methods to solve problems**, bearing in mind **organisational and financial constraints**.

Key phrase 2: All aspects of software production

Not just the **technical process of development**. It also includes **project management** and developing **tools, methods and theories** to support software production.

Engineering means getting results of the **required quality within schedule and budget**, which often needs compromises. A systematic, organised approach is usually best, but a more flexible process suits web-based systems and mobile apps.

Importance of software engineering ★★★ MUST KNOW

1. **Society depends on software**: more and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
2. **It is cheaper in the long run** to use software engineering methods and techniques than to write programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of **changing the software after it has gone into use**.

Software process activities ★★★ MUST KNOW

A **software process** is *a sequence of activities that leads to the production of a software product*. Four fundamental activities are common to all software processes:

1. **Software specification**: customers and engineers define the software that is to be produced and the constraints on its operation.
2. **Software development**: the software is designed and programmed.
3. **Software validation**: the software is checked to ensure that it is what the customer requires.
4. **Software evolution**: the software is modified to reflect changing customer and market requirements.

Systems organise these activities differently: aircraft software is **completely specified before development begins**, while e-commerce specifications and programs are **developed together** (Chapter 2).

> **MEMORY TRICK**
> **S-D-V-E: "Some Developers Value Evolution"**: Specification, Development, Validation, Evolution.

Software engineering vs computer science vs system engineering ★★★ MUST KNOW

- **Computer science** is concerned with the theories and methods that underlie computers and software (theory and fundamentals). It is essential background, as physics is for electrical engineers, but its theories rarely fit large, complex problems.
- **Software engineering** is concerned with the practical problems of producing software, i.e. developing and delivering useful software.
- **System engineering** is concerned with all aspects of developing and evolving complex systems where software plays a major role: hardware development, policy and process design, system deployment *and* software engineering. System engineers specify the system, define its overall architecture and integrate the parts. Software engineering is part of system engineering.

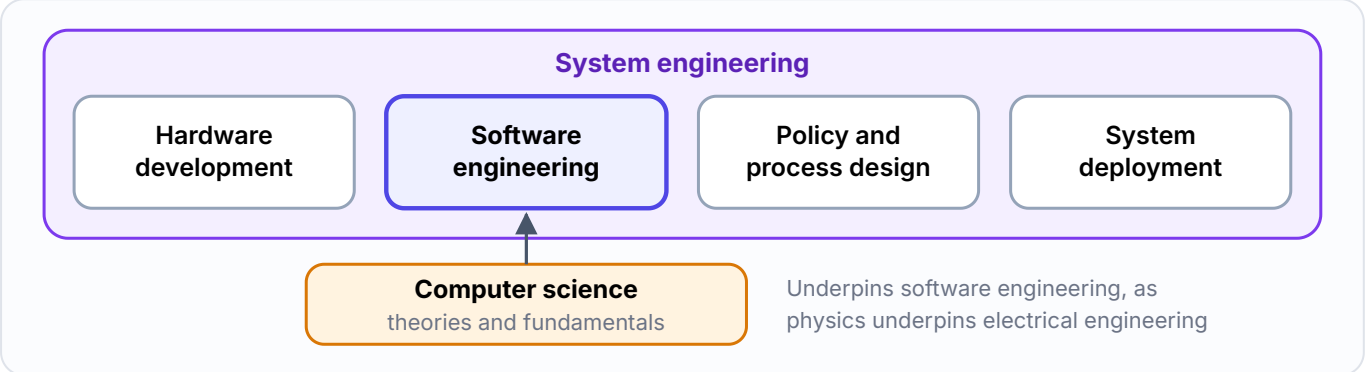


Fig 1.4 Software engineering is one part of system engineering and builds on computer science. Drawn for this guide from Sommerville §1.1.1

1.6 General issues that affect software ★★★ MUST KNOW

No method suits every type of software, but **four related issues** affect many different types:

Issue	Meaning, and the challenge it creates
Heterogeneity	Increasingly, systems are required to operate as **distributed systems across networks** that include different types of computer and mobile devices, and must often integrate with older **legacy systems** in other languages. *Challenge:* dependable software flexible enough to cope.
Business and social change	Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to **change their existing software** and **rapidly develop new software**. *Challenge:* traditional techniques are slow and must deliver value sooner.
Security and trust	As software is intertwined with all aspects of our lives, it is essential that we can **trust** that software, especially remote systems used through web pages or web services. *Challenge:* stop malicious attacks and keep information secure.
Scale	Software has to be developed across a very wide range of scales, from **very small embedded systems** in portable or wearable devices to **Internet-scale, cloud-based systems** that serve a global community.

MEMORY TRICK

"Hungry Bears Steal Snacks": **H**eterogeneity, **B**usiness & social change, **S**ecurity & trust, **S**cale. They match the FAQ's **key challenges**: coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

1.7 Software engineering diversity and application types

★★★ MUST KNOW

- There are many different types of software system, and there is no universal set of software techniques that is applicable to all of them.
- The methods and tools used depend on the **type of application** being developed, the **requirements of the customer** and the **background of the development team**. The type of application is the most significant factor.

The eight application types ★★★ MUST KNOW

Type	What it is	Examples
1. Stand-alone applications	Run on a local computer such as a PC (or as mobile apps). They include all necessary functionality and **do not need a network**.	Office software, CAD programs, photo editors
2. Interactive transaction-based applications	Execute on a **remote computer** and are accessed by users from their own PCs, phones or terminals; often update a large data store in each transaction.	Web applications such as e-commerce; cloud mail
3. Embedded control systems	Software that **controls and manages hardware devices**. Numerically, there are probably more embedded systems than any other type.	Phone software, car anti-lock brakes, microwave oven, insulin pump

Type	What it is	Examples
4. Batch processing systems	Business systems that **process data in large batches**: large numbers of individual inputs create corresponding outputs.	Phone billing, salary payment
5. Entertainment systems	Primarily for **personal use**, intended to entertain the user. Quality of user interaction matters most.	Computer and console games
6. Systems for modelling and simulation	Developed by scientists and engineers to **model physical processes or situations** with many separate, interacting objects; often need high-performance parallel computers.	Scientific simulations of physical processes
7. Data collection systems	**Collect data from their environment using sensors** and send it to other systems for processing; often in hostile places (inside an engine, remote areas).	Wilderness weather station
8. Systems of systems	Systems **composed of a number of other software systems**, some generic (e.g. ERP), some specially written; used by large organisations.	A company-wide system built from an ERP package plus custom systems

MEMORY TRICK

"Some Indian Engineers Build Excellent Models During Summer": **S**tand-alone, **I**nteractive transaction-based, **E**mbedded control, **B**atch, **E**ntertainment, **M**odelling & simulation, **D**ata collection, **S**ystems of systems.

The boundaries are **blurred** (a phone game faces the same power limits as phone software), but each type needs **its own techniques**. A car's safety-critical **embedded system** is burned into ROM, so it needs extensive verification before sale; an **interactive web app** suits iterative delivery from reusable components; a **system of systems** needs its interactions specified in detail in advance.

Software engineering fundamentals ★★★ MUST KNOW

Some fundamental principles apply to **all types of software system**, whatever development techniques are used:

1. **Managed process**: systems should be developed using a managed and understood development process (different processes suit different types of software).
2. **Dependability and performance** are important for all types of system: no failures, available when needed, safe, secure and not wasteful.
3. **Requirements**: understanding and managing the software specification and requirements (what the software should do) are important.
4. **Reuse**: where appropriate, reuse software that has already been developed rather than write new software.

MEMORY TRICK

"Proper Design Requires Reuse": Process, Dependability (& performance), Requirements, Reuse.

1.8 Internet (web-based) software engineering ★★ IMPORTANT

- The **Web is now a platform for running applications**, and organisations are increasingly developing web-based systems rather than local systems.
- **Web services** allow application functionality to be accessed over the web.
- **Cloud computing** is an approach to providing computer services where applications run remotely on the "cloud" (a huge number of linked computers shared by many users). Users do not buy software but pay according to use.

Before the web, business software was **monolithic** (single programs on single computers); now it is deployed on web servers, delivered as **software as a service** (e.g. Google Apps, Office 365), highly distributed and built by reuse. Four changes followed:

Change	Meaning
Software reuse	Reuse is the **dominant approach** for constructing web-based systems: you think about how to assemble them from pre-existing software components and systems.
Incremental and agile development	Web-based systems should be developed and delivered **incrementally**; it is impractical to specify all their requirements in advance.
Service-oriented systems	Software may be built using service-oriented software engineering, where the components are **stand-alone web services**.
Rich interfaces	Interface technologies such as **AJAX and HTML5** support rich interfaces within a web browser.

Web-based systems are complex distributed systems, but the fundamental ideas of software engineering apply to them in the same way as to any other type of system. **Memory trick:** "Reuse It, Serve it Rich" = **R**euse, **I**ncremental, **S**ervice-oriented, **R**ich interfaces.

1.9 Software engineering ethics ★★★ MUST KNOW

- Software engineering involves **wider responsibilities** than simply the application of technical skills.
- Software engineers must behave in an **honest and ethically responsible** way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law; it involves following a set of principles that are morally correct.

Issues of professional responsibility ★★★ MUST KNOW

In these areas, acceptable behaviour is set by **professional responsibility** rather than by laws:

Issue	What the engineer should do	Example
Confidentiality	Respect the confidentiality of employers or clients, **whether or not a formal confidentiality agreement** has been signed.	Not discussing a client's unreleased product
Competence	Do not misrepresent your level of competence or **knowingly accept work outside your competence**.	Not posing as a security expert to win a contract
Intellectual property rights	Know the local laws on **patents, copyright, etc.** and protect the intellectual property of employers and clients.	Not reusing one client's code for another

Issue	What the engineer should do	Example
Computer misuse	Do not use technical skills to misuse other people's computers. Misuse ranges from **trivial** (game playing on an employer's machine) to **extremely serious** (dissemination of viruses).	Not reading others' files with admin access

MEMORY TRICK

"Careful Coders Inspect Code": **C**onfidentiality, **C**ompetence, **I**ntellectual property rights, **C**omputer misuse.

The ACM/IEEE Code of Ethics ★★★ MUST KNOW

- The professional societies in the US (**ACM and IEEE**) have cooperated to produce a code of ethical practice: the *Software Engineering Code of Ethics and Professional Practice*. Other bodies, such as the British Computer Society, publish similar codes.
- Members of these organisations **sign up to the code** of practice when they join.
- The Code contains **eight Principles** related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students.

Rationale: computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society. Software engineers take part (directly or by teaching) in analysing, specifying, designing, developing, certifying, maintaining and testing software, so they have significant opportunities to **do good or cause harm**, or to enable or influence others to. To ensure their efforts are used for good, they must commit to making software engineering a beneficial and respected profession.

Preamble: the **short version** states **aspirations** at a high level; the full version's **clauses** give details. Without aspirations, details become legalistic and tedious; without details, aspirations become high-sounding but empty; together they form a cohesive code. Committed to the **health, safety and welfare of the public**, software engineers shall adhere to eight principles:

Principle	Exact wording (learn this)	In plain words
1. PUBLIC	Software engineers shall act consistently with the public interest.	Public safety first; never hide a harmful defect
2. CLIENT AND EMPLOYER	Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.	Be loyal to them, but not at the public's expense
3. PRODUCT	Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.	Test properly; never ship poor work knowingly
4. JUDGMENT	Software engineers shall maintain integrity and independence in their professional judgment.	Honest opinions; don't sign what you haven't checked
5. MANAGEMENT	Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.	Realistic plans; no pressure to cut ethical corners
6. PROFESSION	Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.	Act so people trust software engineers

Principle	Exact wording (learn this)	In plain words
7. COLLEAGUES	Software engineers shall be fair to and supportive of their colleagues.	Credit others' work; help newcomers
8. SELF	Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.	Keep learning; practise ethics yourself

MEMORY TRICK

Take the principles in pairs: **PC · PJ · MP · CS**, "A **PC** in **PJ**s meets an **MP** from **CS**". That gives **P**ublic–**C**lient, **P**roduct–**J**udgment, **M**anagement–**P**rofession, **C**olleagues–**S**elf. Each pair is one verb: whom you **serve**, what you **deliver**, how you **lead**, how you **grow**.

Ethical dilemmas ★★ IMPORTANT

An **ethical dilemma** is a situation with no clearly right choice, common when people have different views and objectives. The slides give three examples:

1. **Disagreement in principle with the policies of senior management.** Argue your case from within, or resign on principle? When should you report project problems? Too early may be overreacting; too late may make them impossible to fix.
2. **Your employer acts unethically and releases a safety-critical system without finishing the testing** (e.g. falsifies safety validation records). Keep confidentiality, or alert the customer or the public? There are **no absolutes in safety**: disclosure may harm the employer and colleagues, silence may harm others. Weigh the potential damage, its extent and who is affected; go public only if it is very dangerous, and first try to resolve it while respecting the employer's rights.
3. **Participation in the development of military weapons systems or nuclear systems.** Views differ, so **employers and employees should make their views known to each other in advance**.

✎ EXAM TIP

For a 10M "code of ethics" answer: what the code is and who it covers → rationale (2–3 lines) → preamble idea (aspirations + details) → all **eight principles** in their wording → one dilemma. If the question says "professional responsibility", add confidentiality, competence, IPR and computer misuse.

1.10 Case studies used throughout the course ★★ IMPORTANT

Sommerville uses **four different types of system** as examples, because software engineering practice depends on the type of system. Slide 2 says three, but slide 36 introduces four, and later chapters keep returning to all of them.

1. Insulin pump control system ★★ IMPORTANT

An **embedded system** in an insulin pump used by diabetics to maintain blood glucose control. In **diabetes** the pancreas cannot make enough **insulin**; the pump **simulates the pancreas**:

- It **collects data from a blood sugar sensor** and calculates the amount of insulin required to be injected.
- The calculation is based on the **rate of change of blood sugar levels**.
- It **sends signals to a micro-pump** to deliver the correct dose (one pulse = one unit of insulin).
- It is safety-critical: **low** blood sugar can lead to brain malfunctioning, coma and death; **high** blood sugar has long-term consequences such as eye and kidney damage.

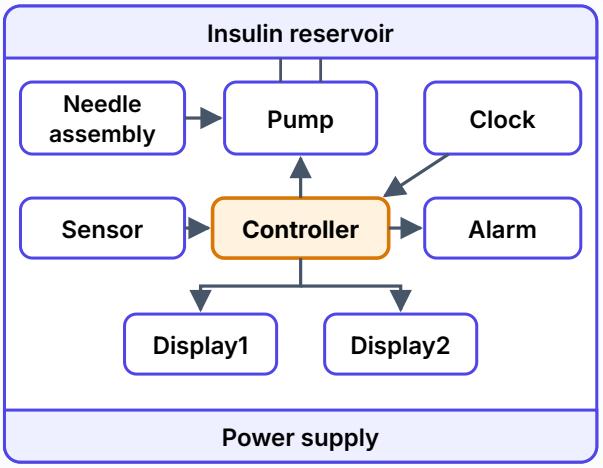


Fig 1.5 Insulin pump hardware architecture; the controller runs the software. Sommerville Fig 1.4 · Your slides, slide 38

A sensor measures a blood parameter (electrical conductivity) that depends on the sugar level; the **controller** works out the dose and pulses the pump. The UML **activity model** shows how the software turns a sensor reading into pump commands:

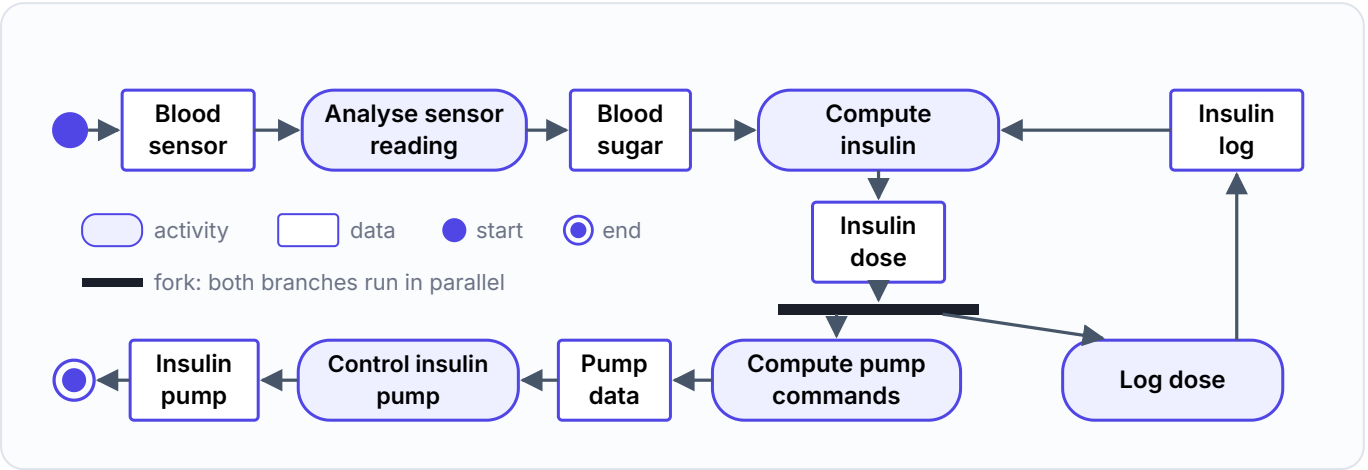


Fig 1.6 Activity model of the insulin pump (UML): sensor → sugar → dose → pump commands, while the dose is also logged. Sommerville Fig 1.5 · Your slides, slide 39

Essential high-level requirements: (1) the system shall be **available** to deliver insulin when required; (2) the system shall perform **reliably** and deliver the correct amount of insulin to counteract the current level of blood sugar. The system must therefore be designed and implemented to ensure that it always meets these requirements.

2. Mentcare: a patient information system for mental health care ★★ IMPORTANT

- A **medical information system** that maintains information about patients suffering from mental health problems and the treatments they have received.
- Most patients do not need hospital treatment but regularly attend **specialist clinics**, which are also held in **local medical practices or community centres**.
- Intended for use in clinics, it uses a **centralised database** of patient information but can also run on a PC at sites without secure network connectivity.
- With secure network access it uses the database; when disconnected it can **download and use local copies** of patient records.

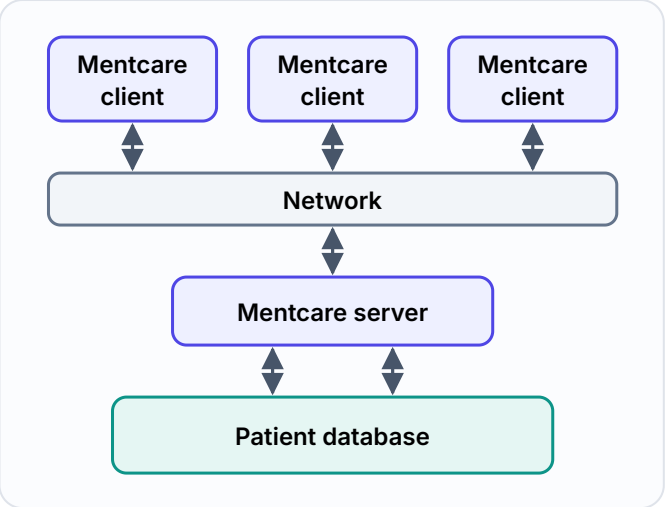


Fig 1.7 Organisation of the Mentcare system. Sommerville Fig 1.6 · Your slides, slide 44

Goals: (1) to generate **management information** that allows health service managers to assess performance against local and government targets; (2) to provide **medical staff** with timely information to support the treatment of patients.

Key features: individual care management: clinicians create, edit and view patient records and history, and data summaries let doctors quickly learn the key problems and prescribed treatments. **Patient monitoring**: the system monitors the records of patients in treatment and issues warnings if possible problems are detected. **Administrative reporting**: monthly management reports on patients treated at each clinic, patients entering and leaving the care system, patients *sectioned* (confined to a secure hospital), and drugs prescribed and their costs.

Concerns: privacy: patient information must be confidential and never disclosed to anyone apart from authorised medical staff and the patient. **Safety**: some patients may become suicidal or a danger to others, so the system should warn medical staff wherever possible. It must also be **available** when needed, or safety may be compromised and correct medication may be impossible to prescribe.

3. Wilderness weather station ★★ IMPORTANT

- A **data collection system** that collects data about weather conditions in remote areas. The government of a country with large areas of wilderness deploys **several hundred** such stations.
- The instruments measure **temperature and pressure, sunshine, rainfall, wind speed and wind direction** (ground and air temperature, barometric pressure, 24-hour rainfall).
- Each instrument is controlled by software that **takes parameter readings periodically** and manages the data collected.

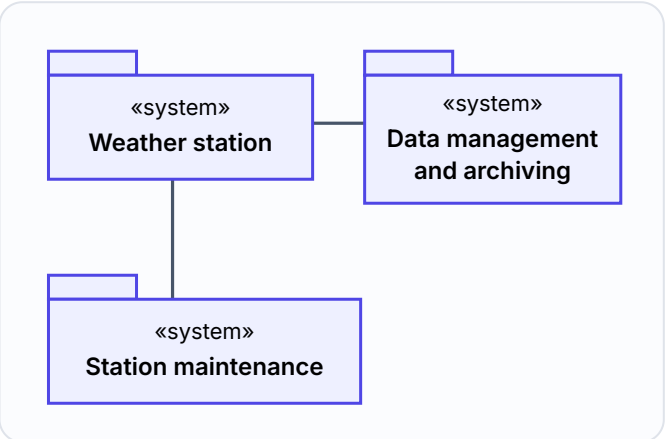


Fig 1.8 The weather station's environment (UML packages with the «system» stereotype). Sommerville Fig 1.7 · Your slides, slide 48

The station belongs to a larger **weather information system** of three systems that exchange information (Fig 1.8). **The weather station system** collects weather data, carries out some initial data processing and transmits it to the data management system. **The data management and archiving system** collects the data from all of the wilderness weather stations, carries out data processing and analysis, and archives the data. **The station maintenance system** communicates by satellite with all wilderness weather stations to monitor their health and provide reports of problems.

Stations are unattended and battery-powered, so the software also needs **additional functionality**: (1) **monitor** the instruments, power and communication hardware and report faults to the management system; (2) **manage the system power**, charging batteries whenever environmental conditions permit but shutting down generators in potentially damaging weather such as high wind; (3) support **dynamic reconfiguration**, where parts of the software are replaced with new versions and backup instruments are switched in after a system failure.

4. iLearn: a digital learning environment ★★ IMPORTANT

- A system to **support learning in schools**: a framework in which general-purpose and specially designed learning tools may be embedded, plus applications geared to the learners' needs.
- The tools are **chosen by teachers and learners**: general applications such as spreadsheets, learning management applications such as a **Virtual Learning Environment (VLE)** for homework submission and assessment, games and simulations.
- It is a service-oriented system: every component is a **replaceable service**. So it can be **updated incrementally** as new services become available, and **rapidly configured** for groups such as very young children who cannot read, or senior students.
- **Services: utility services** give basic application-independent functionality used by other services; **application services** give specific applications (email, conferencing, photo sharing) and educational content (scientific films, historical resources); **configuration services** adapt the environment with a set of application services and define how services are shared between students, teachers and parents.

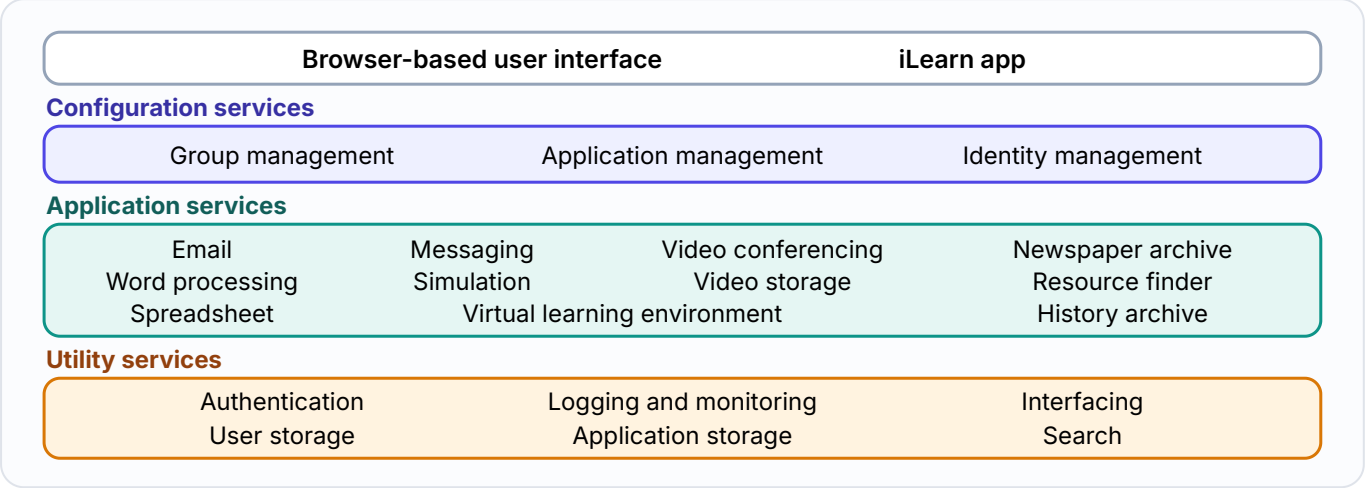


Fig 1.9 The iLearn architecture: user interface, then configuration, application and utility services. Sommerville Fig 1.8 · Your slides, slide 54

Service integration happens at two levels. **Integrated services** offer an **API** (application programming interface) that other services can call, so **direct service-to-service communication** is possible (e.g. authentication). **Independent services** are simply accessed through a **browser interface** and operate independently; information is shared only through user actions such as **copy and paste**, and **re-authentication** may be needed for each one.

Chapter 1 in one page

Definitions

- **Software**: computer programs and associated documentation (plus libraries, support websites, configuration data).
- **Software engineering**: an engineering discipline concerned with all aspects of software production, from early system specification to maintaining the system in use. Key phrases: *engineering discipline* and *all aspects of software production*.
- **Software process**: a sequence of activities that leads to the production of a software product.
- **Generic product**: sold to anyone; developer owns the spec. **Customized (bespoke)**: commissioned by one customer, who owns the spec.

Why SE matters

- Economies depend on software; maintenance costs more than development.
- Failures come from **increasing complexity** and **not using SE methods**.
- Important because society relies on software, and SE is **cheaper in the long run**.

SE vs CS vs system engineering

- CS = theory and fundamentals. SE = practicalities of producing useful software. System engineering = hardware + software + policy/process design + deployment; SE is part of it.

Diagrams to practise

- Insulin pump hardware (Fig 1.5) and activity model (Fig 1.6)
- Mentcare organisation, weather station environment, iLearn architecture (Figs 1.7–1.9)

Lists to learn (with memory tricks)

- **4 attributes** (My Dog Eats Apples): Maintainability, Dependability & security, Efficiency, Acceptability.
- **4 process activities** (S-D-V-E): Specification, Development, Validation, Evolution.
- **4 general issues** (Hungry Bears Steal Snacks): Heterogeneity, Business & social change, Security & trust, Scale.
- **8 application types** (Some Indian Engineers Build Excellent Models During Summer): Stand-alone, Interactive transaction-based, Embedded control, Batch processing, Entertainment, Modelling & simulation, Data collection, Systems of systems.
- **4 fundamentals** (Proper Design Requires Reuse): managed Process, Dependability & performance, Requirements, Reuse.
- **4 web changes** (Reuse It, Serve it Rich): Reuse, Incremental & agile, Service-oriented, Rich interfaces (AJAX, HTML5).
- **4 responsibility issues** (Careful Coders Inspect Code): Confidentiality, Competence, Intellectual property rights, Computer misuse.
- **8 ACM/IEEE principles** (PC · PJ · MP · CS): Public, Client & employer, Product, Judgment, Management, Profession, Colleagues, Self.
- **3 ethical dilemmas**: disagreeing with senior management; employer releasing an untested safety-critical system; military or nuclear work.
- **4 case studies**: insulin pump (embedded), Mentcare (information system), weather station (data collection), iLearn (service-oriented learning environment).

Likely exam questions

2M **Define software and software engineering. State the two key phrases of the definition.**

How to answer → Both definitions word for word (§1.2, §1.5) → "engineering discipline" and "all aspects of software production", one line each.

4M **Differentiate between generic and customized software products, with examples.**

How to answer → Table from §1.3: what, examples, who owns the specification and decides on changes; mention ERP blurring.

6M **Explain the essential attributes of good software.**

How to answer → "Required functionality and performance" first → the four attributes with textbook descriptions → Fig 1.3 → an example each.

4M **What are the fundamental software process activities? Explain each.**

How to answer → Define software process → specification, development, validation, evolution, one line each.

5M **Differentiate software engineering from computer science and from system engineering.**

How to answer → Three short definitions + Fig 1.4: CS = theory; SE = practicalities; SE is part of system engineering.

4M **Why is software engineering important? Why do software projects fail?**

How to answer → Two reasons for importance + two failure causes (increasing complexity; not using SE methods).

6M **Explain the general issues that affect most types of software.**

How to answer → Heterogeneity, business and social change, security and trust, scale: meaning + challenge for each (§1.6).

10M **Explain the types of software applications with examples. Why do they need different techniques?**

How to answer → All eight types with an example → blurred boundaries → car vs web app vs system of systems → the four fundamentals.

6M **How has the web changed software engineering?**

How to answer → Web platform, web services, cloud (pay per use) → reuse, incremental/agile, service-oriented, rich interfaces.

10M **Explain the ACM/IEEE Code of Ethics and the issues of professional responsibility. Give an ethical dilemma.**

How to answer → Four responsibility issues → code: who, rationale, preamble → eight principles in their wording → one dilemma (§1.9).

8M **Describe the insulin pump system with its hardware architecture and activity model. State its essential requirements.**

How to answer → Purpose + why safety-critical → Fig 1.5 → Fig 1.6 → the two high-level requirements.

6M **Explain the Mentcare system (or the weather station, or iLearn) as a case study.**

How to answer → Mentcare: Fig 1.7, goals, features, concerns. Weather station: Fig 1.8, three systems, extra functions. iLearn: services, Fig 1.9, integration.

UNIT 1 · CHAPTER 2

Software Processes

Software process models · Process activities · Coping with change · Process improvement

STUDY TIME
About 4 hours (2 reading + 2 practising the diagrams)

SOURCES
Your Ch 2 slides (58) + Sommerville Ch 2 + Pressman Ch 2 (process models)

EXAM WEIGHT
Very high: "explain with a neat diagram" and "compare the models" are standard questions

AFTER THIS CHAPTER YOU CAN

- Define a software process and its four basic activities
- Compare plan-driven and agile processes
- Draw and explain waterfall, incremental, reuse, V, prototyping and spiral models
- Draw the requirements, design, testing and evolution diagrams
- Explain prototyping and incremental delivery as ways to cope with change
- Tell incremental development and incremental delivery apart
- Explain the improvement cycle and the five CMM levels
- Choose and justify a process model for a case study

2.1 The software process ★★★ MUST KNOW

Every piece of software is produced by people following some sequence of steps: find out what is needed, decide how to build it, build it, check it, and keep changing it afterwards. That organised sequence of steps is the **software process**.

> **DEFINITION — LEARN THIS LINE**
>
> **Software process** — a structured set of activities required to develop a software system. (Textbook: a set of related activities that leads to the production of a software system.)

The four fundamental activities ★★★ MUST KNOW

There are many different software processes, but all of them involve these four activities:

Activity	What it means (slide wording)
1. Specification	Defining what the system should do (its functionality and the constraints on its operation).
2. Design and implementation	Defining the organisation of the system and implementing the system (textbook: *software development*).
3. Validation	Checking that it does what the customer wants.
4. Evolution	Changing the system in response to changing customer needs.

Each activity has sub-activities (e.g. requirements validation, architectural design, unit testing), and processes also contain supporting activities such as configuration management and project planning.

> **MEMORY TRICK**
> **S-D-V-E** — "**S**ome **D**evelopers **V**alue **E**volution": Specification, Design & implementation, Validation, Evolution.

Software process models and process descriptions ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Software process model** — an abstract representation of a process. It presents a description of a process from some particular perspective.

A process model is also called a **Software Development Life Cycle (SDLC) model**; the very general models are called **process paradigms**. Being a simplified view from one perspective, a model shows only *part* of the process (e.g. an activity model shows activities and their order, but not who does them). Companies adapt these general frameworks into their own processes.

When we describe a process we usually talk about its **activities** (e.g. specifying a data model, designing a user interface) and the **ordering** of these activities. A process description may also include:

Element	Meaning	Example
Products	The outcomes of a process activity.	Architectural design produces a model of the software architecture.
Roles	The responsibilities of the people involved in the process.	Project manager, configuration manager, programmer.
Pre- and post-conditions	Statements that are true before and after a process activity has been enacted or a product produced.	Pre: customer has approved all requirements. Post: the architecture models have been reviewed.

Plan-driven and agile processes ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Plan-driven processes** — processes where all of the process activities are planned in advance and progress is measured against this plan.
>
> **Agile processes** — planning is incremental, and it is easier to change the process to reflect changing customer requirements.

Point	Plan-driven	Agile
Planning	All activities planned in advance; progress measured against the plan	Incremental and continual as the software is developed
Change	Costly: plans and documents must be redone	Easy to change the process to follow the customer
Requirements	Complete specification before design starts	Specified informally for each increment just before it is built

Point	Plan-driven	Agile
Documents & testing	Detailed design documents; testing driven by test plans and an independent test team	Minimum documentation; design kept in the code; tests written with the requirements
Suits	Safety-critical, embedded, large multi-site systems (e.g. waterfall, V-model)	Business systems and products with rapidly changing requirements (Chapter 3)

★ KEY POINT

There are no right or wrong software processes — no single process suits every system. The choice depends on the type of software, the customer's requirements and the developers' skills: a safety-critical system needs a very structured process with detailed records; a business system with fast-changing requirements suits a flexible agile process. In practice, most practical processes include elements of both plan-driven and agile approaches.

2.2 Software process models (Sommerville) ★★★ MUST KNOW

Model	Key idea (slide wording)	Plan-driven or agile?
The waterfall model	Separate and distinct phases of specification and development.	Plan-driven
Incremental development	Specification, development and validation are interleaved.	May be plan-driven or agile
Integration and configuration	The system is assembled from existing configurable components.	May be plan-driven or agile

In practice, **most large systems are developed using a process that incorporates elements from all of these models**: well-understood parts are built waterfall-style or bought and configured, hard-to-specify parts are developed incrementally, and components are reused throughout (the architecture, though, must be planned up front). Typically, safety-critical software uses waterfall, software products are incremental, and business systems are configured from existing systems. Read once: the **Rational Unified Process (RUP)** tries to combine all the general models; its phases are inception, elaboration, construction and transition.

The waterfall model ★★★ MUST KNOW

IN SIMPLE WORDS

Water in a waterfall falls from one ledge to the next and never flows back up. Each phase "cascades" into the next: finish and sign off the requirements, then design, then code, then test, then deliver.

The first published process model (Royce, 1970), taken from large military systems engineering; also called the **software life cycle**. It is **plan-driven**: you plan and schedule all activities before starting development.

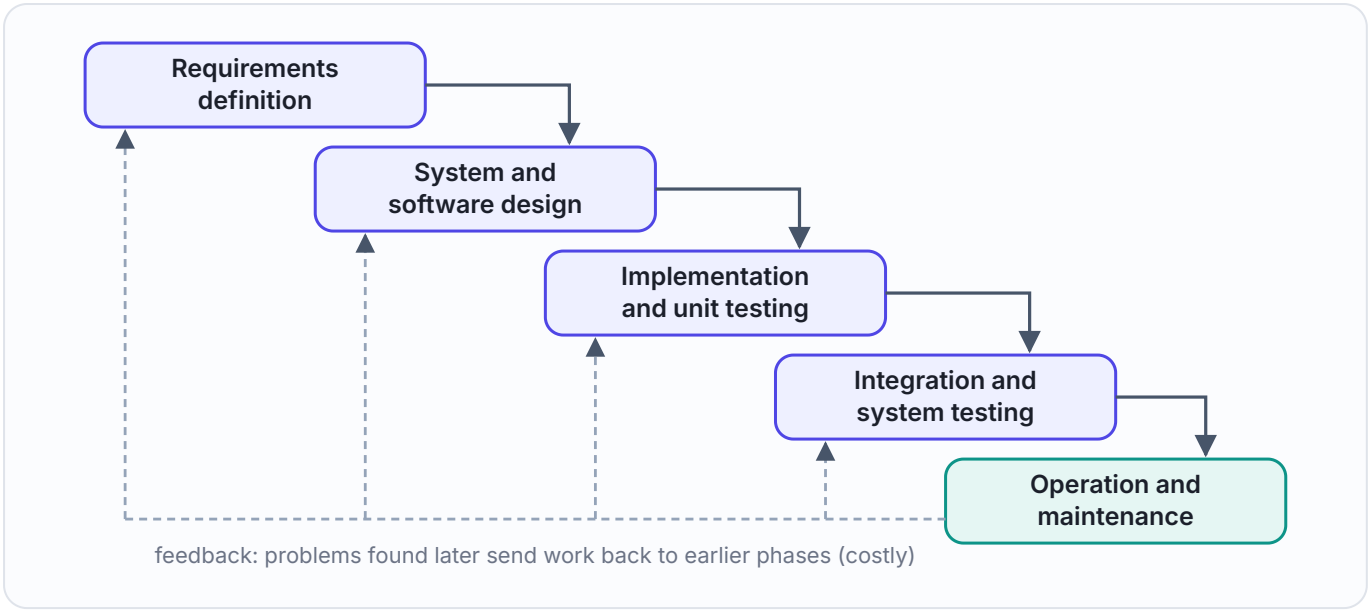


Fig 2.1 The waterfall model. Each phase must finish (and be signed off) before the next begins. Sommerville Fig 2.1 · Your slides, slide 8

1. **Requirements analysis and definition** — the system's services, constraints and goals are established with users and defined in detail as the system specification.
2. **System and software design** — requirements are allocated to hardware or software and an overall architecture is established; software design identifies the fundamental software abstractions and their relationships.
3. **Implementation and unit testing** — the design is realised as programs or program units; each unit is verified against its specification.
4. **Integration and system testing** — units are integrated and tested as a complete system against the requirements, then delivered to the customer.
5. **Operation and maintenance** — normally the *longest* phase: the system is used; maintenance corrects errors not found earlier, improves the implementation and adds new services.

The result of each phase is one or more approved ("*signed-off*") documents, and a phase has to be complete before moving onto the next. In real software, phases overlap and feed information back (dashed lines in Fig 2.1). The main drawback is the difficulty of accommodating change after the process is underway.

> **MEMORY TRICK**
>
> **R-S-I-I-O** — "**R**ain **S**lides **I**nto **I**cy **O**ceans": Requirements, System design, Implementation, Integration, Operation.

When to use it: embedded systems (hardware is inflexible, so software decisions can't be delayed); critical systems (need safety and security analysis of a complete specification and design); and **large systems engineering projects developed at several sites** or by partner companies, where its plan-driven nature helps coordinate the work. Avoid it when requirements change quickly and the team can communicate informally. Read once: **formal system development** (e.g. the B method) is a waterfall variant that transforms a mathematical specification into code; used only for very critical systems.

✎ EXAM TIP

10 marks: diagram with feedback lines → one line per phase → "plan-driven, signed-off documents" → 3 advantages, 3 problems → when appropriate, with an example (insulin pump).

ADVANTAGES

- ✓ Simple, systematic; easy to plan and manage
- ✓ Signed-off documents make progress visible
- ✓ Plan-driven nature coordinates work at several sites
- ✓ Complete specification allows safety/security analysis

PROBLEMS

- ✗ Inflexible partitioning into distinct stages makes it difficult to respond to changing customer requirements
- ✗ Only appropriate when requirements are well understood and changes limited — few business systems have stable requirements
- ✗ Requirements get frozen too early, so the system may not do what users want
- ✗ Pressman: customers can't state all requirements up front; no working version until late; team members wait in "blocking states"

Incremental development ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Incremental development — develop an initial implementation, get feedback from users, and evolve the software through several versions until the required system is developed. Specification, development and validation are interleaved, with rapid feedback across activities.

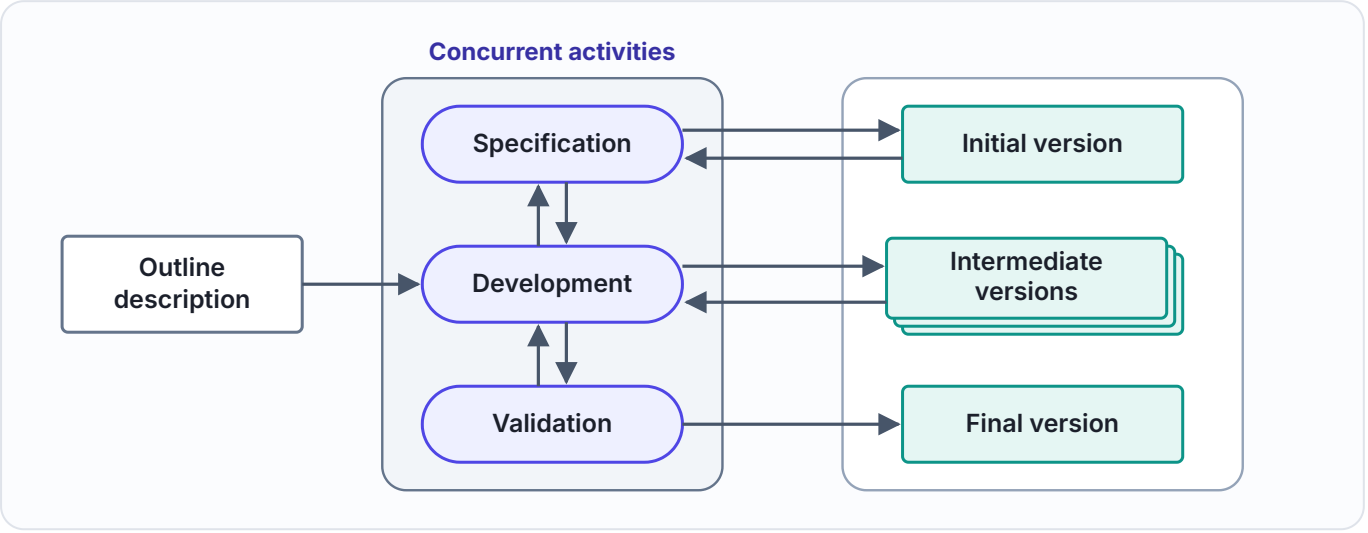


Fig 2.2 Incremental development: activities run concurrently and produce a series of versions. Sommerville Fig 2.2 · Your slides, slide 11

Now the **most common approach** for application systems and software products, and the basis of agile methods. It can be plan-driven (increments identified in advance), agile (later increments depend on progress and priorities) or a mix. Early increments hold the **most important or urgent functionality**, so the customer evaluates the system early; if it is wrong, only the current increment changes.

IN SIMPLE WORDS

Like writing an essay in drafts: write a rough version, show it, improve it, show it again — each draft (increment) adds to the last.

BENEFITS

- ✓ **Cost of accommodating changing requirements is reduced** — far less analysis and documentation to redo than with waterfall.
- ✓ **Easier to get customer feedback** — customers comment on demonstrations and see how much has been implemented.
- ✓ **More rapid delivery and deployment** of useful software — customers gain value earlier than with waterfall.

PROBLEMS

- ✗ **The process is not visible** — managers need regular deliverables to measure progress; documenting every quickly-built version is not cost-effective.
- ✗ **System structure tends to degrade** as increments are added. Unless time and money is spent on *refactoring*, regular change corrupts the structure and further changes become difficult and costly.

These problems are worst for large, long-lifetime systems built by several teams, which need a stable architecture planned in advance.

⚠ WATCH OUT

Incremental **development** does not mean every increment is delivered to the customer — versions may only be *shown* for comment. Deploying increments for real use is **incremental delivery** (Section 2.5.2).

Integration and configuration (reuse-oriented SE) ★★★ MUST KNOW

IN SIMPLE WORDS

Like furnishing a room with ready-made furniture you assemble and adjust, instead of making every piece from raw wood.

- Based on **software reuse**: systems are integrated from existing components or application systems, sometimes called **COTS (commercial-off-the-shelf)** systems.
- Reused elements may be **configured** to adapt their behaviour and functionality to a user's requirements.
- Reuse is now the **standard approach for building many types of business system** (covered in depth in Sommerville Chapter 15).

Types of reusable software: (1) **stand-alone application systems** (COTS) configured for a particular environment; (2) **collections of objects** developed as a package to be integrated with a component framework such as .NET or J2EE (textbook: Java Spring); (3) **web services** developed to service standards and available for remote invocation.

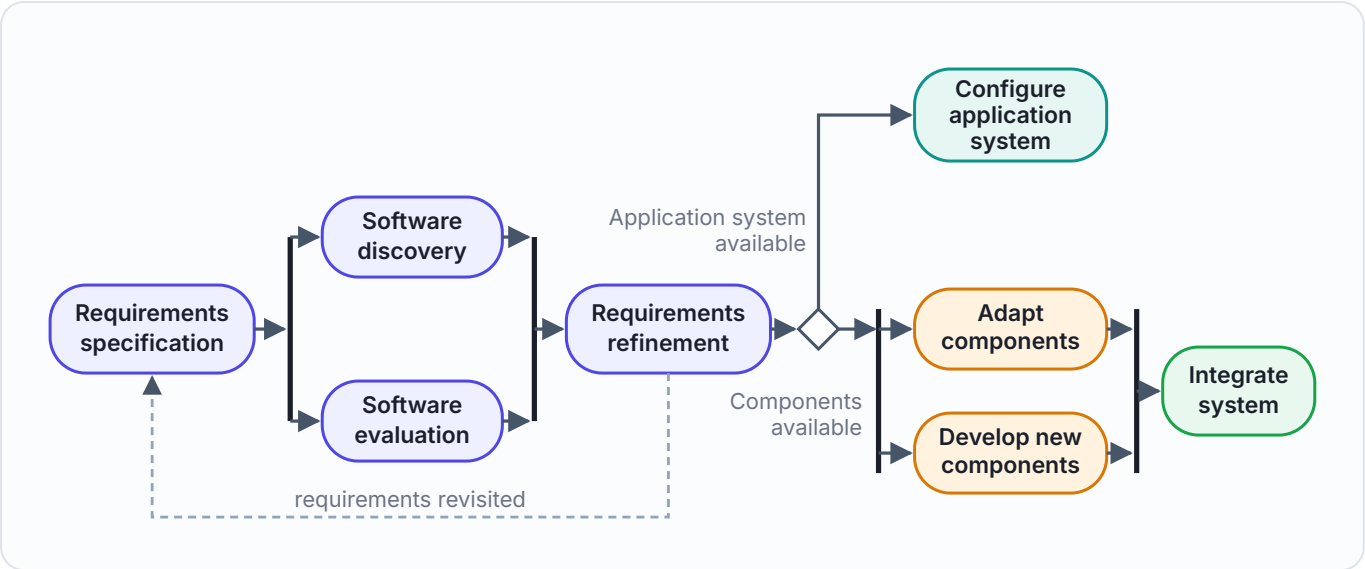


Fig 2.3 Reuse-oriented software engineering. Thick bars split or join parallel paths; the diamond is a decision. Sommerville Fig 2.3 · Your slides, slide 16

1. **Requirements specification** — brief initial requirements: essential requirements and desirable features.
2. **Software discovery and evaluation** — search for components/systems providing the functionality and evaluate whether they meet the essential requirements.
3. **Requirements refinement** — modify the requirements to fit the components found; if impossible, search again (so requirements engineering is repeated).
4. **Application system configuration** — if an off-the-shelf system meets the requirements, configure it to create the new system.
5. **Component adaptation and integration** — otherwise, modify reusable components, develop new ones, and integrate them.

ADVANTAGES

- ✓ Reduced costs and risks as less software is developed from scratch
- ✓ Faster delivery and deployment of the system

DISADVANTAGES

- ✗ Requirements compromises are inevitable, so the system may not meet the real needs of users
- ✗ Loss of control over evolution of reused system elements

EXAMPLE

iLearn (digital learning environment for schools) is built this way: its application services (email, conferencing, spreadsheets, a virtual learning environment) are bought or freely available, and configuration services adapt it to each school.

2.3 More process models (from Textbook 2, Pressman)

★★★ MUST KNOW

These models are not in Sommerville's chapter or your slides (except the V-model, slide 30) but are commonly examined. Pressman calls them **prescriptive process models**, and labels his diagrams with five generic **framework activities**: **Communication** (requirements gathering), **Planning** (estimating, scheduling,

tracking, risk analysis), **Modeling** (analysis, design), **Construction** (code, test) and **Deployment** (delivery, support, feedback). His waterfall ("classic life cycle") is these five in sequence.

The V-model ★★★ MUST KNOW

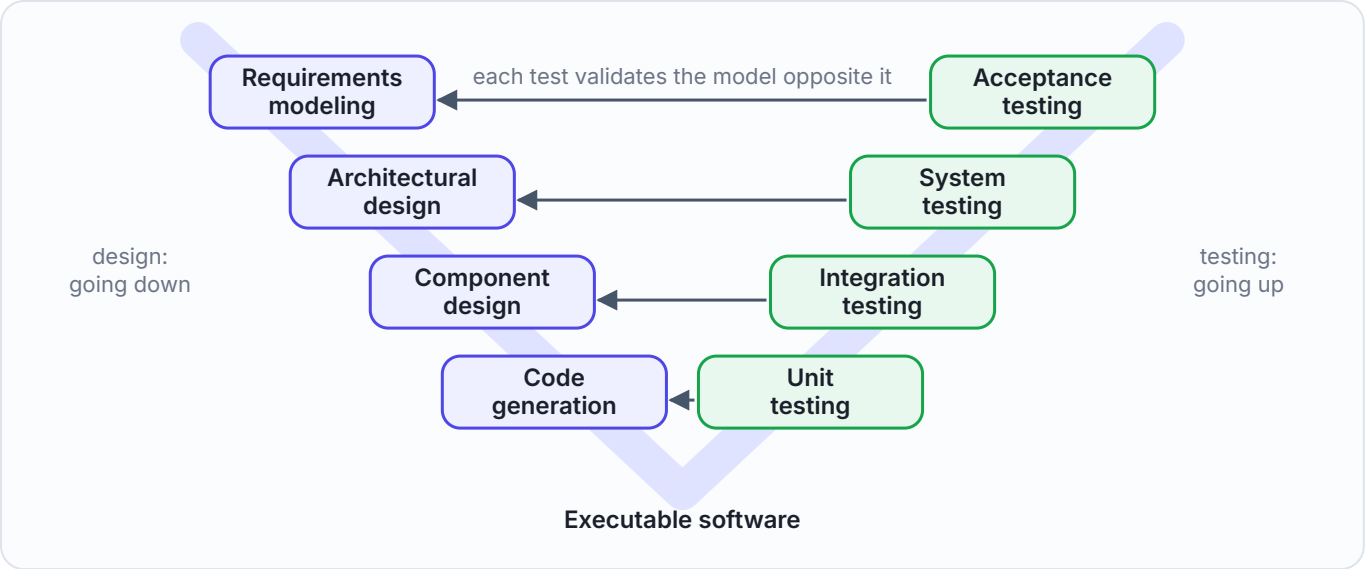


Fig 2.4 The V-model as drawn by Pressman. Pressman Fig 2.4 (From Textbook 2)

- A **variation of the waterfall model** showing how quality-assurance (verification and validation) actions relate to the earlier communication, modeling and construction actions.
- Going **down the left side**, requirements are refined into more detailed technical representations; after code is generated, the team goes **up the right side**, running tests that validate each model made on the way down.
- There is *no fundamental difference* between the classic life cycle and the V-model; the V simply visualises where V&V is applied. Pros: tests planned early, clear stage-to-test link. Cons: as inflexible as waterfall; working software late.
- Sommerville's version (your slide 30) adds the **test plans** — Fig 2.10 in Section 2.4.3.

Pressman's incremental model (read once) applies linear sequences in a staggered way over calendar time; each delivers an operational increment, the first usually a *core product* (e.g. a word processor: basic editing first, then advanced editing, then spell-checking, then page layout). Because each increment is delivered to the customer, it matches Sommerville's **incremental delivery**.

Evolutionary model 1: the prototyping paradigm ★★★ MUST KNOW

Evolutionary models are iterative: each iteration produces a more complete version. Use **prototyping** when the customer has general objectives but no detailed requirements, or the developer is unsure of an algorithm's efficiency, an operating system's adaptability or the form of the user interaction. It can be a stand-alone model but is more often a *technique* used inside other models.

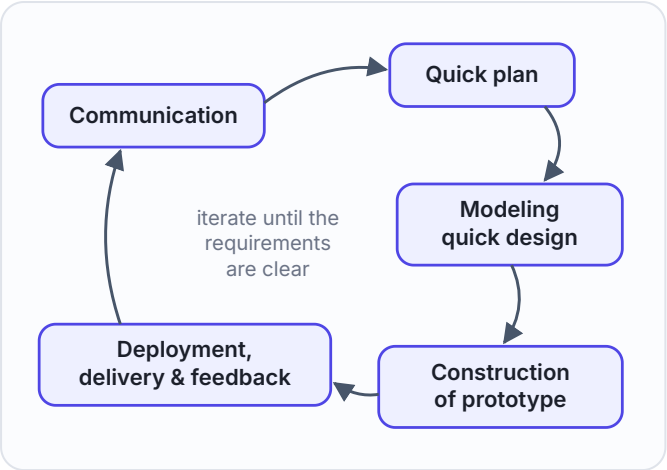


Fig 2.5 The prototyping paradigm. Pressman Fig 2.6 (From Textbook 2)

1. **Communication** — define overall objectives, known requirements and areas needing definition.
2. **Quick plan** — plan an iteration quickly.
3. **Modeling (quick design)** — of what users will see: screen layouts, output formats.
4. **Construction of prototype** — build quickly, reusing fragments and tools.
5. **Deployment, delivery & feedback** — stakeholders evaluate; feedback refines the requirements. Iterate.

A **throwaway** prototype is discarded after use (Brooks: "plan to throw one away"); an **evolutionary** prototype slowly grows into the real system. **Strengths:** clarifies fuzzy requirements; users get an early feel for the system; developers build something at once. **Problems:** (1) stakeholders see an apparently working version, unaware it is held together haphazardly, and demand "a few fixes" to ship it; (2) quick implementation compromises (wrong language, inefficient algorithm) become a permanent part of the system.

> ★ KEY POINT
>
> Define **the rules of the game at the beginning**: everyone agrees the prototype only defines requirements; it is then discarded (at least in part) and the real software is engineered for quality. Sommerville's prototyping (slides 37–41) is in Section 2.5.1.

Evolutionary model 2: the spiral model (Boehm) ★★★ MUST KNOW

> IN SIMPLE WORDS
>
> Like climbing a spiral staircase: on each round you pass the same points again (communicate, plan and analyse risks, design, build, deliver), but each round takes you higher — to a more complete system — and you check for risks every time.

Proposed by **Barry Boehm (1988)**: an evolutionary, **risk-driven** model that couples the *iterative nature of prototyping* with the *controlled, systematic aspects of the waterfall model*. Its two distinguishing features: (1) a **cyclic approach** that grows the system's definition and implementation while **decreasing its degree of risk**; (2) **anchor point milestones** that secure stakeholder commitment to feasible, mutually satisfactory solutions.

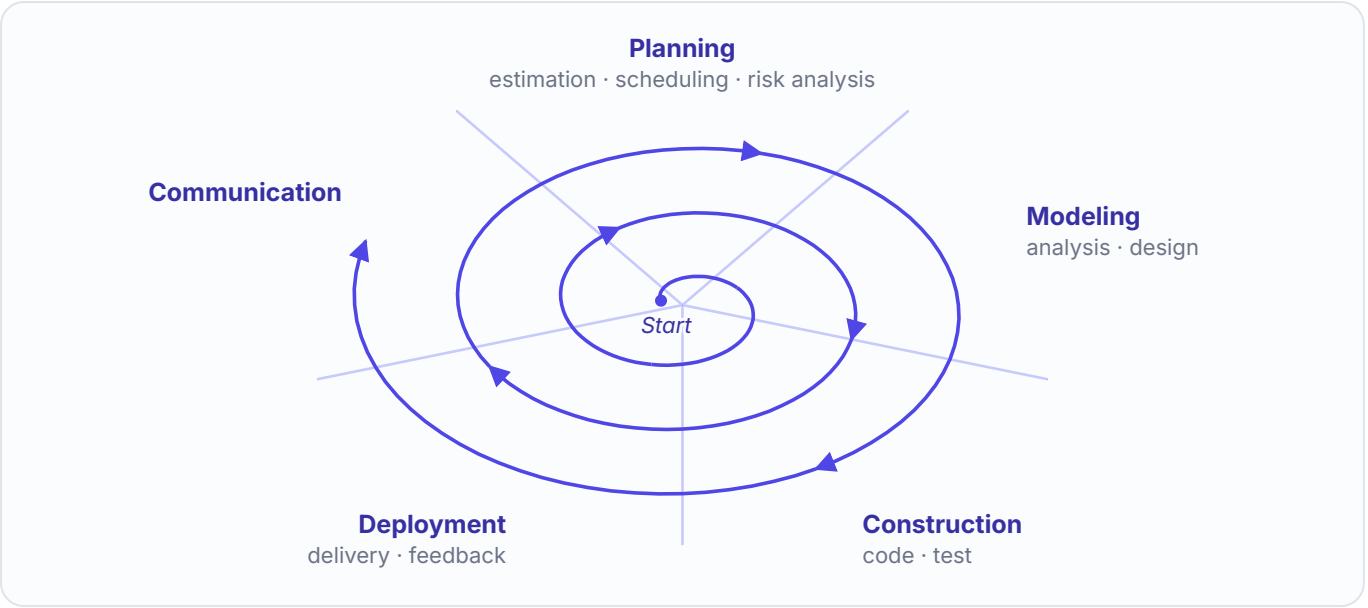


Fig 2.6 A typical spiral model. Start at the centre and move clockwise; each circuit yields a more complete version, and risk is analysed on every circuit. Pressman Fig 2.7 (From Textbook 2)

- The team moves **clockwise from the centre** through the framework activities; **risk is considered on each revolution** and anchor point milestones are noted on each pass.
- Early circuits produce a *specification* or *prototype*; later circuits produce progressively more complete versions. Each pass through planning adjusts cost, schedule and the number of iterations.
- Sommerville: **each loop represents a phase** (innermost — feasibility, next — requirements definition, next — system design, ...). It combines change avoidance with change tolerance and has **explicit risk management**.
- It can cover the whole life of the software (concept development → new product development → product enhancement) until the software is retired.

Also seen in exam answers: Boehm's original drawing (used in older Sommerville editions) splits each loop into four sectors — objective setting; risk assessment and reduction; development and validation; planning. Either drawing earns marks if labelled.

ADVANTAGES

- ✓ Realistic approach for large-scale systems
- ✓ Developer and customer understand and react to risks at each level
- ✓ Prototyping used as risk reduction at any stage
- ✓ Systematic stepwise approach inside an iterative framework

DISADVANTAGES

- ✗ Hard to convince customers (in contracts) it is controllable
- ✗ Demands considerable risk-assessment expertise
- ✗ If a major risk is missed, problems will occur
- ✗ Poor fit for fixed budgets: cost revised every circuit

Read once — the concurrent model (concurrent engineering): all activities exist at the same time, each in a *state* (inactive, under development, awaiting changes, under review, under revision, baselined, done); *events* trigger transitions, e.g. a requirements error found during design moves requirements analysis from "done" to "awaiting changes". It gives an accurate picture of a project's current state.

2.4 Process activities ★★★ MUST KNOW

Real software processes are **inter-leaved sequences of technical, collaborative and managerial activities** with the overall goal of specifying, designing, implementing and testing a software system (usually with tool support).

> ★ KEY POINT
> The four basic activities — specification, development, validation and evolution — are **organised differently in different processes**: in the waterfall model they are **in sequence**, whereas in incremental development they are **interleaved**.

2.4.1 Software specification ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
> **Software specification** — the process of establishing what services are required and the constraints on the system's operation and development. This is **requirements engineering**: the process of developing a software specification.

It is a **critical stage**: mistakes here cause later problems in design and implementation. A short, cheap **feasibility study** may come first (is there a need, and is it technically and financially realistic?). The goal is an agreed **requirements document** at two levels: *user requirements* for customers and end-users, and detailed *system requirements* for developers.

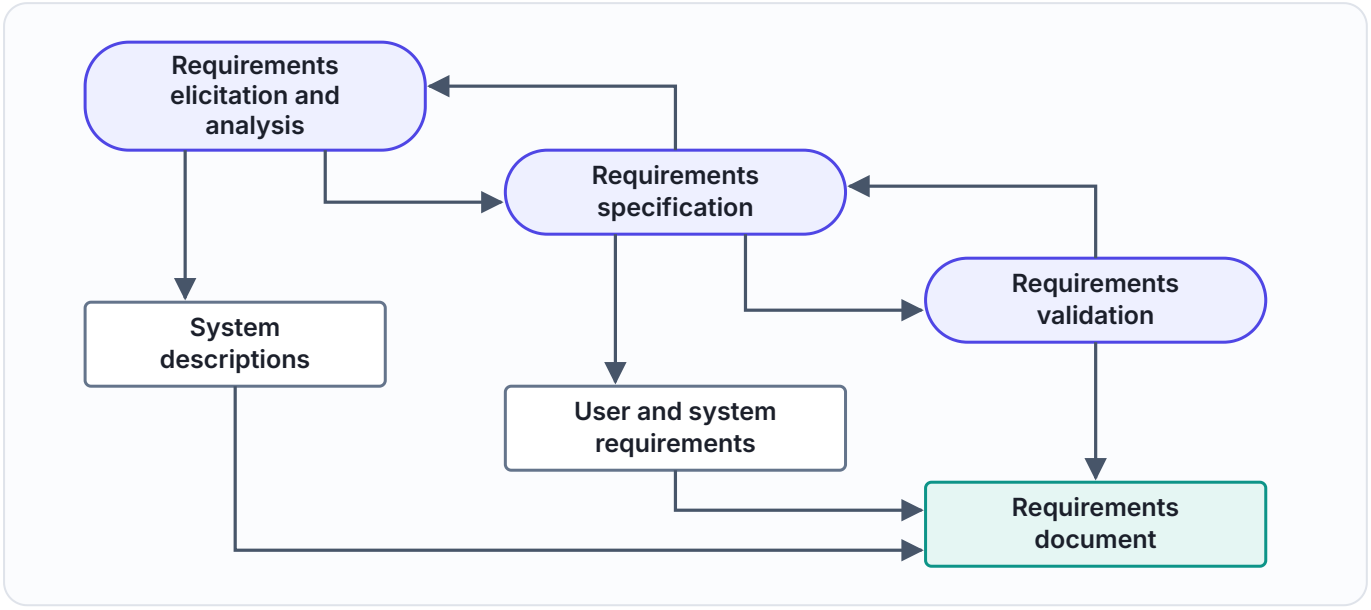


Fig 2.7 The requirements engineering process. Rounded boxes are activities; square boxes are products (documents).
Sommerville Fig 2.4 · Your slides, slide 21

Activity	Slide point	Textbook detail
Requirements elicitation and analysis	What do the system stakeholders require or expect from the system?	Observe existing systems, talk to users and procurers, task analysis; may use models and prototypes.
Requirements specification	Defining the requirements in detail.	Write the gathered information as user requirements and system requirements.

Activity	Slide point	Textbook detail
Requirements validation	Checking the validity of the requirements.	Check realism, consistency and completeness; correct the errors found.

The activities are **interleaved** (back-arrows in Fig 2.7). In **agile** methods specification is not separate: requirements are written informally for each increment just before it is built. *Example (insulin pump):* "The system shall be available to deliver insulin when required" is a high-level user requirement.

2.4.2 Software design and implementation ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
> **Design and implementation** — the process of converting the system specification into an executable system.

Software design: design a software structure that realises the specification (structure, data, interfaces between components, sometimes algorithms). **Implementation**: translate this structure into an executable program. The two are **closely related and may be inter-leaved**; in agile development design is informal (whiteboards) with no formal design documents.

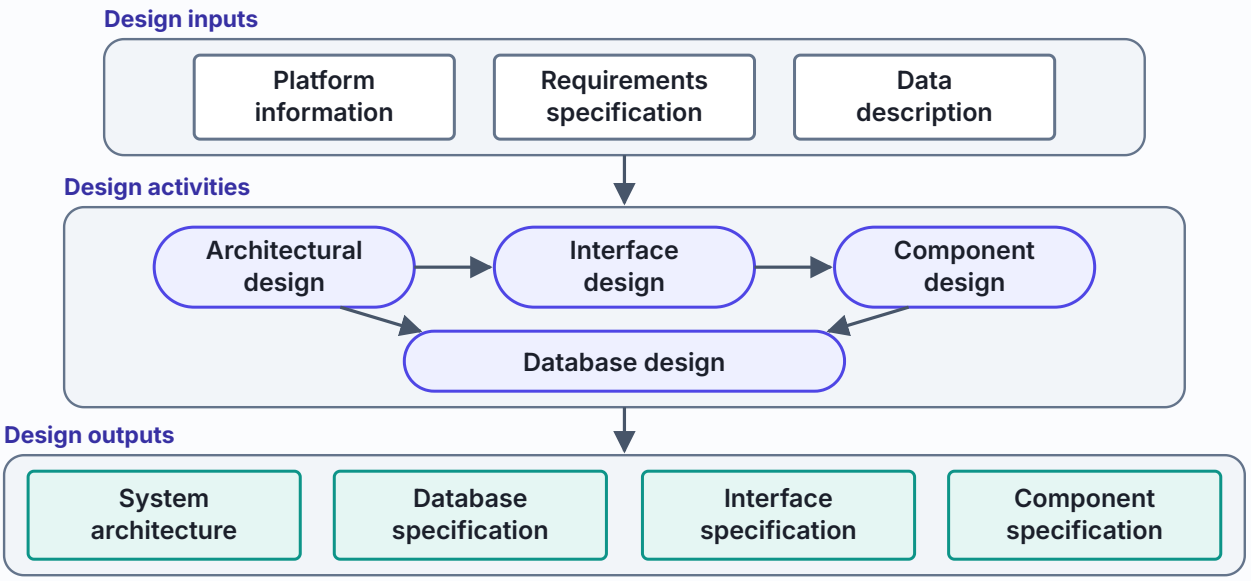


Fig 2.8 A general model of the design process. Sommerville Fig 2.5 · Your slides, slide 24

Inputs: platform information (the operating system, database, middleware and other systems the software runs with), the requirements specification, and a data description. **Outputs:** detailed documents for critical systems, design diagrams in model-driven development, or just the code in agile development.

Design activity	What you do
Architectural design	Identify the overall structure of the system, the principal components (sub-systems or modules), their relationships and how they are distributed.
Database design	Design the system data structures and how these are to be represented in a database.
Interface design	Define the interfaces between system components — unambiguously, so a component can be used without knowing its implementation.

Design activity	What you do
Component selection and design	Search for reusable components; if unavailable, design how the new component will operate.

The activities are interleaved and interdependent, so design rework is inevitable; they also vary (a real-time system adds *timing design* and may have no database).

System implementation ★★ IMPORTANT

- The software is implemented either by **developing a program or programs** or by **configuring an application system**. Design and implementation are interleaved for most systems.
- **Programming is an individual activity with no standard process** (some programmers start with familiar components, others leave them till last).
- **Debugging** is the activity of finding program faults and correcting these faults — by forming and testing hypotheses about the program's behaviour, often with interactive debuggers.

> ⚠ WATCH OUT
>
> **Testing ≠ debugging.** Testing establishes the *existence* of defects; debugging *locates and corrects* them.

2.4.3 Software validation ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
>
> **Verification and validation (V & V)** — is intended to show that a system conforms to its specification and meets the requirements of the system customer.

In short, validation checks that the system conforms to its specification *and* meets the real needs of its users. V & V involves **checking and review processes** and **system testing**. System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system. **Testing is the most commonly used V & V activity.** Except for small programs, systems are tested in stages, not as one unit.

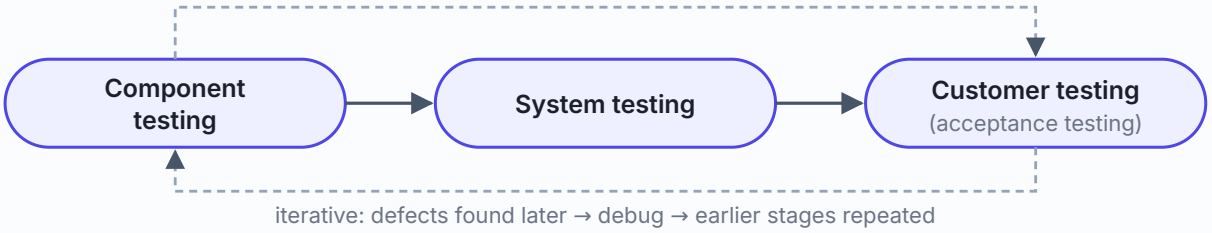


Fig 2.9 Stages of testing (slide 28 labels the last stage "Acceptance testing"). Sommerville Fig 2.6 · Your slides, slide 28

Stage	Slide points	Textbook detail
Component testing	Individual components are tested independently; components may be functions or objects or coherent groupings of these.	Done by the developers; tools such as JUnit rerun tests automatically.

Stage	Slide points	Textbook detail
System testing	Testing of the system as a whole. Testing of emergent properties is particularly important.	Finds interaction and interface errors; checks functional and non-functional requirements.
Customer testing	Testing with customer data to check that the system meets the customer's needs.	Last stage before use; may reveal requirements problems. For products: **beta testing** by selected users.

Emergent properties belong to the whole system, not to any one part (e.g. overall reliability, performance) — so they can only be tested after integration.

Testing phases in a plan-driven software process (V-model) ★★★ MUST KNOW

In a plan-driven process (e.g. for critical systems), an independent team of testers works from **test plans** developed from the system specification and design. The test plans are the **link between development and testing**; the V-model shows the validation activity that corresponds to each stage of the waterfall model.

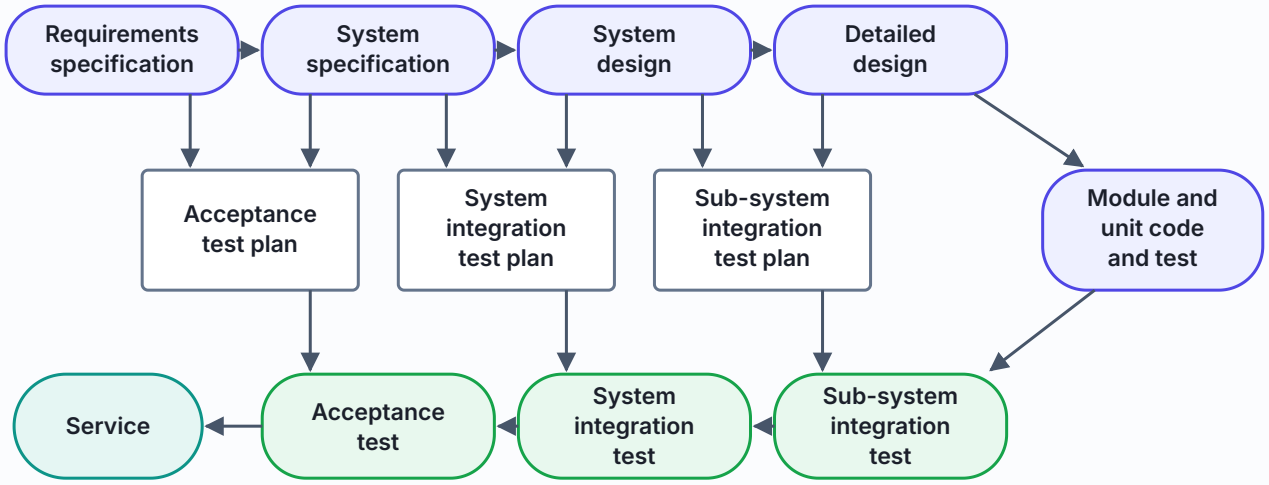


Fig 2.10 Testing phases in a plan-driven software process (V-model). Square boxes are test plans written during development and used later in testing. Your slides, slide 30 · Sommerville Fig 2.7

Read it as a V lying on its side: development runs along the top to *module and unit code and test*; testing returns along the bottom to *service*. Requirements + system specification → **acceptance test plan**; system specification + system design → **system integration test plan**; system design + detailed design → **sub-system integration test plan.** (The 10th-edition textbook says "customer test plan", "component design" and "component code and test" — either labelling is correct.)

2.4.4 Software evolution ★★★ MUST KNOW

- **Software is inherently flexible and can change** — far more cheaply than hardware, even after delivery.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change — **it must evolve to remain useful**.
- Although there has been a demarcation between development and evolution (maintenance), this is **increasingly irrelevant as fewer and fewer systems are completely new**; development and maintenance form one continuum.

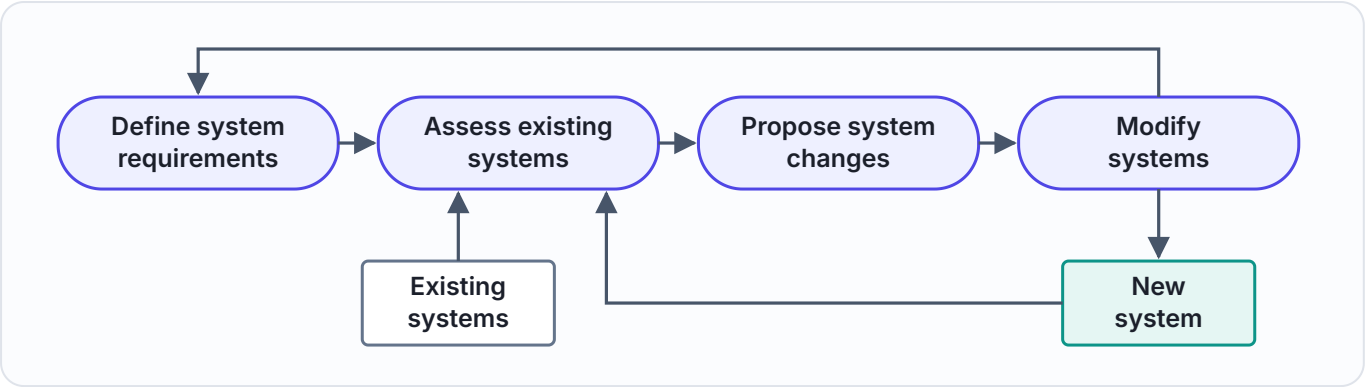


Fig 2.11 System evolution: the new system becomes an existing system that will itself be changed. Sommerville Fig 2.8 · Your slides, slide 32

2.5 Coping with change ★★★ MUST KNOW

Change is inevitable in all large software projects: business changes lead to new and changed system requirements; new technologies open up new possibilities for improving implementations; changing platforms require application changes. Change leads to **rework** (redoing completed work), so the costs of change include both rework (e.g. re-analysing requirements) and the costs of implementing new functionality.

	Change anticipation	Change tolerance
Meaning	The process includes activities that can anticipate possible changes **before significant rework is required**.	The process is designed so that changes can be accommodated at **relatively low cost**.
Example	A prototype system shows key features to customers, who refine their requirements before costly development.	Incremental development: changes go into increments not yet developed; if impossible, only a single increment (a small part of the system) is altered.
Technique	**System prototyping**	**Incremental delivery** (also gives change *avoidance*); refactoring

So there are two ways of coping with changing requirements: **system prototyping** — a version of the system or part of it is developed quickly to check the customer's requirements and the feasibility of design decisions (supports change anticipation); and **incremental delivery** — increments are delivered to the customer for comment and experimentation (supports both change avoidance and change tolerance). Processes structured for iterative development and delivery let changes be made without disrupting the system as a whole.

2.5.1 Software prototyping ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Prototype** — an initial version of a system used to demonstrate concepts and try out design options.

A prototype can be used in

1. **Requirements engineering** — to help with requirements elicitation and validation.
2. **Design** — to explore options and develop a UI design (rapid prototyping with users is the only sensible way to develop user interfaces).
3. **Testing** — to run *back-to-back tests*: the same tests run on the prototype and the system, and the outputs are compared.

Benefits of prototyping

1. Improved system usability
2. A closer match to users' real needs
3. Improved design quality
4. Improved maintainability
5. Reduced development effort

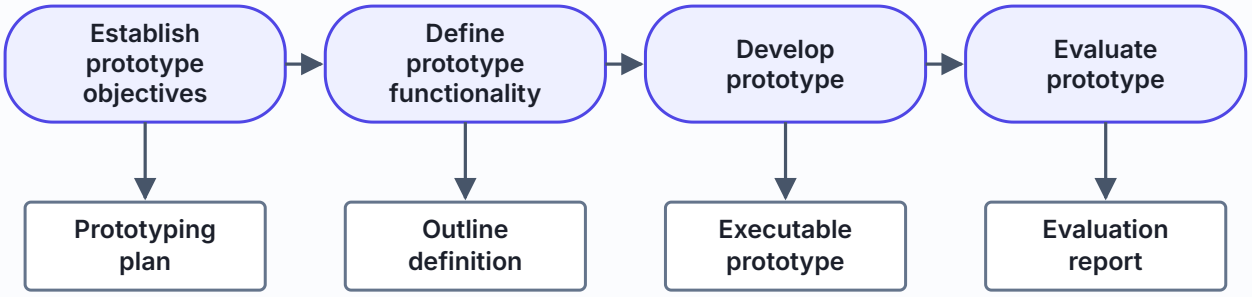


Fig 2.12 The process of prototype development: four activities, each with its output. Sommerville Fig 2.9 · Your slides, slide 39

Objectives must be explicit (e.g. develop the UI, validate functional requirements, or demo to managers) — one prototype rarely meets all of them. **Functionality**: decide what to leave out. **Evaluation**: train users and let them settle into normal use; note that users may not use a prototype the way they will use the final system. **Prototype development** (slide 40):

- May be based on rapid prototyping languages or tools.
- May involve leaving out functionality: focus on areas of the product that are **not well understood**; error checking and recovery may not be included; focus on **functional** rather than non-functional requirements such as reliability and security.

Throw-away prototypes

Prototypes should be **discarded after development**, as they are not a good basis for a production system:

1. It may be impossible to tune the system to meet non-functional requirements.
2. Prototypes are normally undocumented.
3. The prototype structure is usually degraded through rapid change.
4. The prototype probably will not meet normal organisational quality standards.

? QUICK CHECK

Your manager wants to ship your impressive prototype as the production system. What do you say?

Answer: Give the four throw-away reasons: it cannot be tuned for performance, security or reliability; it has no documentation; its structure is degraded, so it is hard to maintain; it fails quality standards. Use it as a specification and build the real system properly.

2.5.2 Incremental delivery ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Incremental delivery — rather than deliver the system as a single delivery, the development and delivery is broken down into increments, with each increment delivering part of the required functionality.

User requirements are **prioritised** and the highest priority requirements are included in early increments. Once the development of an increment is started, **its requirements are frozen**, though requirements for later increments can continue to evolve. Each finished increment is installed in the customer's working environment and integrated with earlier ones.

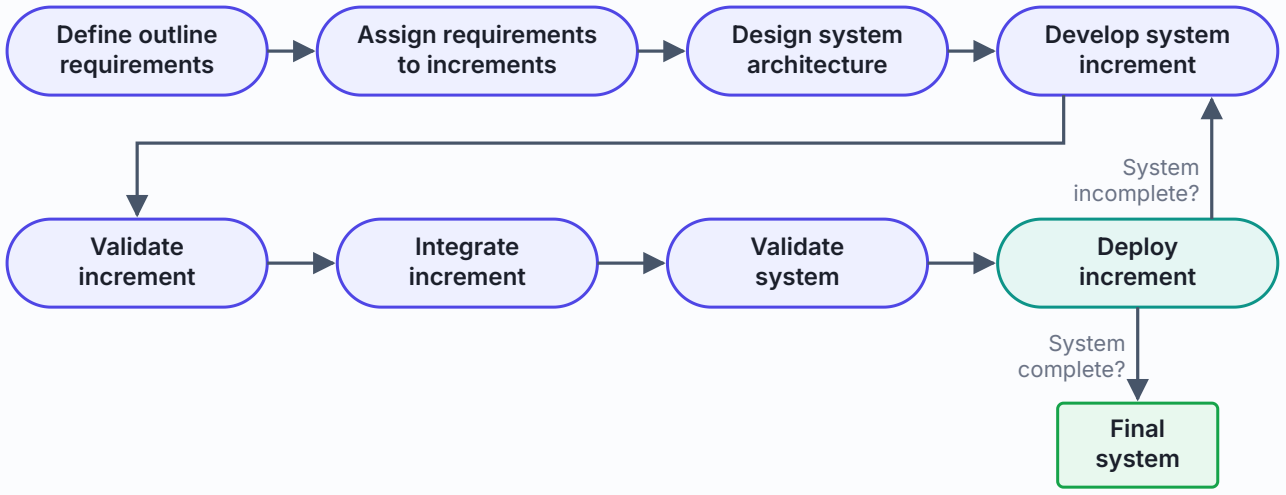


Fig 2.13 Incremental delivery. The "System incomplete?" loop repeats once per increment. Sommerville Fig 2.10 · Your slides, slide 44

ADVANTAGES

- ✓ **Customer value can be delivered with each increment**, so system functionality is available earlier.
- ✓ **Early increments act as a prototype** to help elicit requirements for later increments (and they are part of the real system — no relearning).
- ✓ **Lower risk of overall project failure.**
- ✓ **The highest priority system services tend to receive the most testing.**

PROBLEMS

- ✗ Most systems need **basic facilities used by different parts**; as requirements are not detailed until an increment is implemented, these common facilities are hard to identify.
- ✗ In iterative processes the specification is developed with the software; this **conflicts with the procurement model** of many organisations, where the complete specification is part of the contract.
- ✗ Hard for **replacement systems** — users won't accept less than the old system.

Incremental development vs incremental delivery ★★★ MUST KNOW

Point	Incremental development	Incremental delivery
Each increment	Develop the system in increments and **evaluate each increment** before proceeding to the next	**Deploy** each increment for use by end-users

Point	Incremental development	Incremental delivery
Evaluated by	User or **customer proxy** (someone representing the customer), via demos	Real end-users in their working environment — a **more realistic evaluation** of practical use
Typical use	The **normal approach used in agile methods**	New systems whose users have time to experiment
Main difficulty	Process not visible; structure degrades	**Difficult for replacement systems**, as increments have less functionality than the system being replaced

★ KEY POINT

Incremental **development** is about **how you build** (in pieces, with feedback). Incremental **delivery** is about **how you release** (every piece goes into real use). Delivery implies incremental development, but not the other way round.

2.6 Process improvement ★★★ MUST KNOW

Many software companies have turned to **software process improvement** as a way of enhancing the quality of their software, reducing costs or accelerating their development processes.

DEFINITION — LEARN THIS LINE

Process improvement — understanding existing processes and changing these processes to increase product quality and/or reduce costs and development time.

	Process maturity approach	Agile approach
Focus	Improving process and project management and introducing good software engineering practice	Iterative development and the reduction of overheads in the software process
Key idea	The **level of process maturity** reflects the extent to which good technical and management practice has been adopted in the organisation's processes	The primary characteristics of agile methods are **rapid delivery of functionality** and **responsiveness to changing customer requirements**
Goal / style	Product quality and process predictability; plan-driven, adds "overhead" activities	Best processes have the lowest overheads; minimal formality and documentation

The process improvement cycle ★★★ MUST KNOW

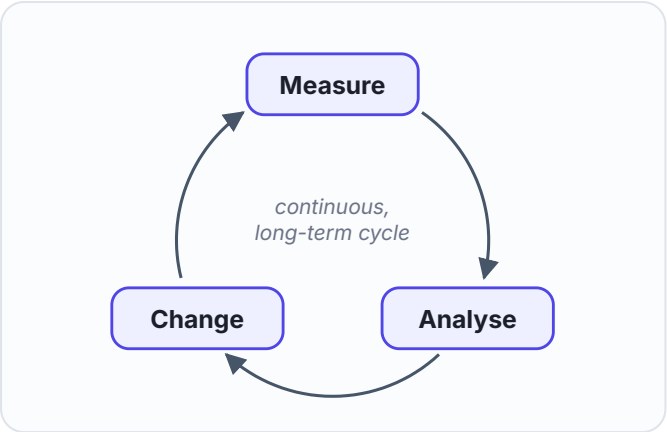


Fig 2.14 The process improvement cycle. Sommerville Fig 2.11 · Your slides, slide 50

1. **Process measurement** — measure one or more attributes of the process or product. These measurements form a **baseline** to decide if improvements have been effective.
2. **Process analysis** — assess the current process; identify **weaknesses and bottlenecks**. Process models (**process maps**) may be developed.
3. **Process change** — propose and introduce changes for the weaknesses; the cycle resumes to collect data on their effectiveness.

Process measurement: wherever possible, **quantitative process data should be collected** — but where organisations do not have clearly defined process standards this is very difficult, as you don't know what to measure; a process may have to be defined first. Measurements should be used to **assess** improvements, but should not **drive** them: the improvement driver should be the organisational objectives.

Process metric	Example
Time taken for process activities to be completed	Calendar time or effort to complete an activity or process
Resources required for processes or activities	Total effort in person-days
Number of occurrences of a particular event	Number of defects discovered

Capability maturity levels — the SEI capability maturity model ★★★ MUST KNOW

In the late 1980s the **Software Engineering Institute (SEI)** proposed a model of process capability maturity (Humphrey, 1988) so that the U.S. Department of Defense could assess defence contractors and limit contracts to mature ones. Its five maturity levels essentially correspond to the use of good software engineering practice. The model later evolved into CMMI (slide 54 names).

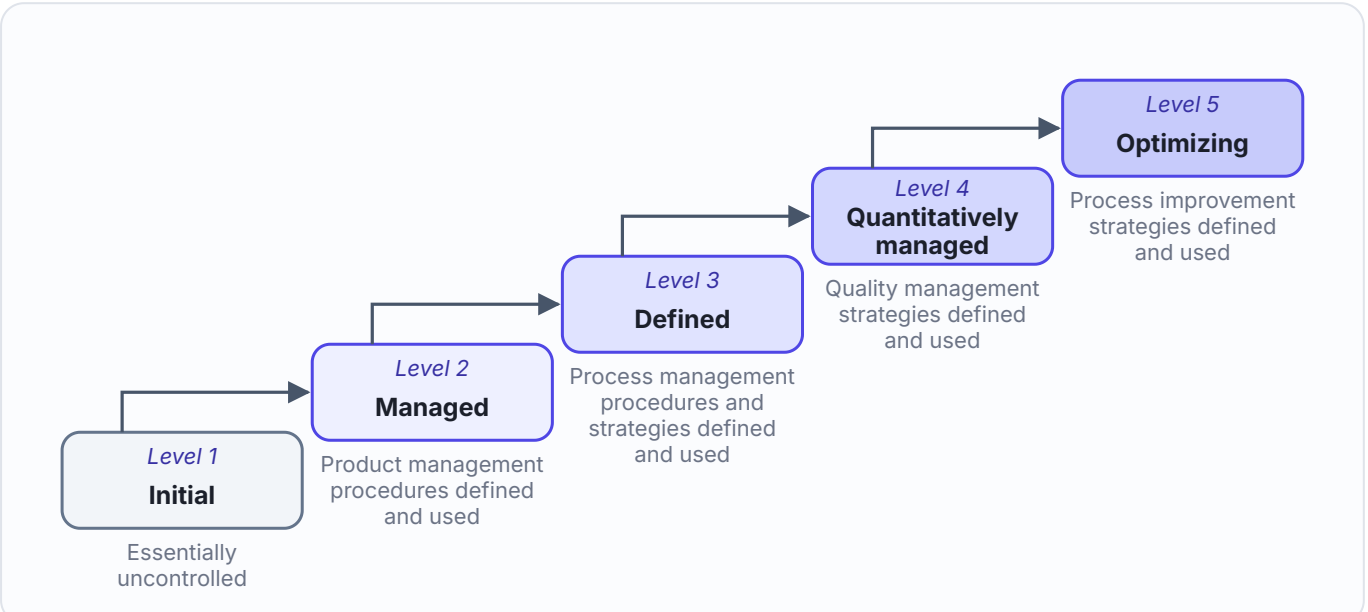


Fig 2.15 Capability maturity levels as a staircase (box names from slide 54; grey meanings from slide 55). Sommerville Fig 2.12 · Your slides, slides 54–55

No.	SEI CMM (slide 55)	CMMI (slide 54)	Meaning (slide 55)	Textbook detail
1	**Initial**	Initial	Essentially uncontrolled	Process goals satisfied; scope of work set out and communicated to the team.
2	**Repeatable**	Managed	Product management procedures defined and used	Policies say when each process is used; documented project plans; resource management and process monitoring.
3	**Defined**	Defined	Process management procedures and strategies defined and used	Standard organisational processes adapted per project; process measurements kept for improvement.
4	**Managed**	Quantitatively managed	Quality management strategies defined and used	Statistical/quantitative methods control sub-processes using process and product measurements.
5	**Optimising**	Optimizing	Process improvement strategies defined and used	Measurements drive improvement; trends analysed; processes adapted to business needs.

MEMORY TRICK

SEI levels bottom to top: **I**nitial, **R**epeatable, **D**efined, **M**anaged, **O**ptimising — "**I** **R**eally **D**o **M**ean **O**rder". CMMI renames only levels 2 and 4.

Impact and limits: the maturity work focused attention on processes and led to significant improvements in capability, but it has **too much overhead for small companies** and **maturity estimation with agile processes is difficult**, so mainly large companies use it. (Pressman also names CBA IPI, SPICE (ISO/IEC 15504) and ISO 9001:2000 for Software as assessment standards — read once.)

2.7 Comparing all the process models ★★★ MUST KNOW

Model	Core idea	Best when	Strengths	Weaknesses	Example system
Waterfall	Separate, sequential, signed-off phases; plan-driven	Stable, well-understood requirements; embedded, critical, multi-site projects	Easy to manage; documents at every phase; supports safety analysis	Inflexible to change; working software late; requirements frozen early	Insulin pump; antilock braking system
V-model	Waterfall bent into a V; each design stage has a test plan and a test level	Critical, plan-driven systems with independent testers	Tests planned early; clear stage-to-test traceability	As inflexible as waterfall; little early working software	Insulin pump; avionics control software
Incremental development	Spec, development and validation interleaved; series of versions	Changing requirements; business systems and products	Cheaper change; easy feedback; early useful software	Process not visible; structure degrades	iLearn; Mentcare; a mobile app

Model	Core idea	Best when	Strengths	Weaknesses	Example system
Incremental delivery	Prioritised increments deployed to users one by one	New system; early value needed; users can experiment	Early value; increments act as prototypes; lower risk; key services tested most	Common facilities hard to find; clashes with fixed contracts; bad for replacements	Mentcare clinics; Pressman's word processor
Integration & configuration	Configure and integrate COTS systems, components, web services	Suitable reusable software exists; business systems	Lower cost and risk; faster delivery	Requirements compromises; no control over evolution of reused parts	iLearn services; an ERP or accounting system
Prototyping	Quick version to learn requirements or try designs, then thrown away	Fuzzy requirements; critical UI; technical doubt	Usability; closer match to real needs; fewer late changes	Pressure to ship the prototype; shortcuts stay	Mentcare user interface; a VR system
Spiral (Boehm)	Risk-driven loops; each circuit a more complete version	Large, complex, high-risk, long-lived systems	Explicit risk management; realistic for large systems	Needs risk expertise; hard to fix by contract; heavy for small jobs	A national weather information system

⚠ **WATCH OUT**

Never write "waterfall is bad, agile is good". Every model suits some systems. Marks come from linking the choice to the **system's characteristics**: requirements stability, criticality, hardware dependence, size and spread of teams, available reusable software, and risk.

2.8 Comparative analysis on the case studies (lab) ★★★ MUST KNOW

For the lab (and the "suggest a model" exam question) use five steps: **(1)** characterise the system (type, criticality, hardware, users, size, replacing an old system?); **(2)** list deciding criteria; **(3)** rate each model; **(4)** recommend one model with reasons tied to the characteristics; **(5)** say what you borrow from other models — real processes are hybrids.

Case study	Waterfall	V-model	Incr. dev.	Incr. delivery	Reuse	Prototyping	Spiral
Insulin pump	✓✓	✓✓	✗	✗	✗	✓ (requirements)	✓
Mentcare	✗	✗	✓✓	✓	✓	✓✓ (UI)	✓
Weather station	✓✓	✓	✗	✗	✓ (data store)	✓	✓✓
iLearn	✗	✗	✓✓	✓✓	✓✓	✓ (UI)	✗

✓✓ strong fit · ✓ useful inside a hybrid · ✗ poor fit. These are reasoned suggestions; another choice also scores if justified from the system's characteristics.

System and characteristics	Recommended	Why (tie each reason to a characteristic)	Not suitable / watch out
Insulin pump — embedded medical device; sensor + pump hardware; safety-critical	Waterfall with V-model test plans (formal methods for the most critical parts)	Embedded: hardware is fixed early, so software decisions can't wait. Critical: a complete specification and design are needed for safety analysis; regulators need signed-off documents; independent testing against test plans.	Incremental delivery (no "half a pump"); agile's light documentation. Use a prototype only to explore requirements.
Mentcare — patient information system for mental-health clinics; many user types; privacy, security, legal duties	Incremental development + UI prototyping	Many users' needs are hard to fix up front and will change; clinicians give feedback per increment (care management first, then patient monitoring, then reports); prototyping gives usability and a closer match to real needs.	Specify privacy and legal requirements carefully up front. If it replaces an old system, incremental delivery is hard.
Wilderness weather station — unattended, battery-powered, satellite link, hostile environment; part of a larger weather information system	Plan-driven (waterfall/V) for station software, inside a spiral for the whole system	Software depends on instrument, power and comms hardware fixed early; several sub-systems and partners to coordinate; high risks (power, communication, damage) suit the spiral's explicit risk analysis.	Pure agile/incremental delivery. Reuse existing database software for data archiving.
iLearn — service-based learning environment for ages 3–18; bought or free application services; replaceable services	Integration and configuration + incremental (agile) delivery	Most functionality exists as services, so reuse cuts cost and time; configuration services adapt each school's version; needs vary and evolve; teachers give quick feedback; not safety-critical.	Requirements compromises and loss of control over external services' evolution.

? QUICK CHECK

Practice (Sommerville Exercise 2.1): choose a model for (a) an antilock braking system, (b) a virtual-reality system to support software maintenance, (c) a university accounting system replacing an existing one, (d) an interactive travel planning system.

Answer: (a) Waterfall/V-model — embedded and safety-critical. (b) Incremental development with prototyping — uncertain requirements and interface. (c) Plan-driven, or configure an accounting package — requirements known from the old system; incremental delivery is hard for replacements. (d) Incremental development — user-facing, requirements change with feedback.

Chapter 2 in one page

Definitions

- **Software process**: structured set of activities required to develop a software system.
- **Process model**: abstract representation of a process from a particular perspective.
- **Plan-driven**: all activities planned in advance; progress measured against the plan. **Agile**: incremental planning; easy to change.
- **Prototype**: initial version used to demonstrate concepts and try out design options.
- **Process improvement**: understand and change processes to raise quality and/or reduce cost and time.

Lists to write in full

- **Activities**: specification, design & implementation, validation, evolution.
- **Process description**: products, roles, pre-/post-conditions.
- **Waterfall**: requirements; system & software design; implementation & unit test; integration & system test; operation & maintenance.
- **Reusable software**: COTS apps; object collections (.NET/J2EE); web services.
- **Reuse stages**: spec; discovery & evaluation; refinement; configuration; adaptation & integration.
- **RE**: elicitation & analysis; specification; validation.
- **Design**: architectural, database, interface, component selection & design.
- **Testing**: component, system, customer (acceptance).
- **Prototyping** uses: RE, design, back-to-back testing. Benefits: usability, real needs, design quality, maintainability, less effort.

Comparisons

- **Waterfall vs incremental**: sequential vs interleaved; change cost; feedback; visibility.
- **Incr. development vs delivery**: evaluate with proxy vs deploy to real users.
- **Anticipation vs tolerance**: prototyping vs incremental.
- **Maturity vs agile improvement**: good practice vs low overhead.
- **Testing vs debugging**: find defects vs locate and fix them.

Numbers and names

- Incremental development: **3 benefits, 2 problems**. Incremental delivery: **4 advantages, 2 problems**.
- Throw-away reasons: NFRs untunable; undocumented; degraded structure; below standards.
- Improvement cycle: **measure → analyse → change**. Metrics: **time, resources, events**.
- SEI CMM: **Initial, Repeatable, Defined, Managed, Optimising**.
- Spiral: Boehm 1988, risk-driven. Waterfall: Royce 1970.

Diagrams to practise

- Waterfall · incremental · reuse · both V-models · prototyping paradigm · spiral
- RE process · design process · testing stages · evolution
- Prototype development · incremental delivery · improvement cycle · CMM staircase

Case studies

- Pump → waterfall + V · Mentcare → incremental + prototyping · Weather station → plan-driven in a spiral · iLearn → reuse + incremental.

Likely exam questions

4M **Define software process and software process model. List the four fundamental activities and what process descriptions include.**

How to answer → Slide definitions word for word; four activities; products, roles, pre-/post-conditions (2.1).

5M **Differentiate between plan-driven and agile processes.**

How to answer → Two definitions → 5-row table → "most processes combine both; no right or wrong process".

10M **Explain the waterfall model with a neat diagram. Discuss its problems and when it is appropriate.**

How to answer → Fig 2.1 → 5 phases → main drawback → problems → embedded, critical, multi-site systems.

8M **Explain incremental development with a diagram, with its benefits and problems.**

How to answer → Fig 2.2 → interleaving, versions → 3 benefits, 2 problems with sub-points.

8M **Explain reuse-oriented software engineering (integration and configuration).**

How to answer → COTS, configuration → 3 types of reusable software → Fig 2.3 → 5 stages → pros and cons.

10M **Explain Boehm's spiral model with a neat diagram. List its advantages and disadvantages.**

How to answer → Fig 2.6 → risk-driven circuits, anchor point milestones, each loop a phase → pros, cons.

6M **Explain the V-model. How are test plans linked to development in a plan-driven process?**

How to answer → Fig 2.10 (slide 30) and the three test plans; mention Pressman's V (Fig 2.4) pairs.

8M **With neat diagrams, explain the requirements engineering process and the general model of design.**

How to answer → Fig 2.7 + three RE activities; Fig 2.8 + inputs, four design activities, outputs.

6M **Explain the stages of testing. Distinguish testing from debugging.**

How to answer → V & V definition → Fig 2.9 → three stages → testing vs debugging.

6M **Why is change inevitable? Explain change anticipation and change tolerance.**

How to answer → Three causes → rework → anticipation vs tolerance table → the two ways of coping.

10M **What is a prototype? Explain prototype development with a diagram, and why throw-away prototypes are discarded.**

How to answer → 3 uses, 5 benefits → Fig 2.12 → slide 40 points → 4 reasons (Pressman's Fig 2.5 is a bonus).

8M **Explain incremental delivery with a diagram. How does it differ from incremental development?**

How to answer → Priorities, freezing → Fig 2.13 → 4 advantages, problems → comparison table.

10M **Explain the process improvement cycle and the SEI capability maturity levels.**

How to answer → Two approaches → Fig 2.14 + three activities → Fig 2.15 → five levels with meanings.

10M **Suggest and justify process models for the insulin pump, Mentcare, the weather station and iLearn.**

How to answer → For each: characteristics → model → reasons → what to borrow (2.8).

UNIT 1 · CHAPTER 3

Agile Software Development

Agile methods · Extreme Programming & agile techniques · Agile project management (Scrum) · Scaling agile

STUDY TIME
About 3 hours

SOURCES
Your slides (Ch 3) + Sommerville Ch 3

EXAM WEIGHT
Very high: manifesto, principles, XP practices and the Scrum sprint cycle are standard 5–10 mark questions

AFTER THIS CHAPTER YOU CAN

- Explain why rapid (agile) development is needed
- Differentiate plan-driven and agile development with a diagram
- Quote the Agile Manifesto and explain the five agile principles
- Draw the XP release cycle and list all ten XP practices
- Explain user stories, refactoring, test-first development and pair programming
- Define every Scrum term and explain the Scrum sprint cycle
- State the benefits of Scrum and the needs of distributed Scrum
- Discuss when agile or plan-driven development is the better choice

3.1 Rapid and agile development ★★★ MUST KNOW

Rapid software development ★★★ MUST KNOW

- **Rapid development and delivery** is now often the most important requirement for software systems. Businesses may even accept lower quality or fewer features to get essential software deployed quickly.
- Businesses operate in a **fast-changing environment**, so it is practically **impossible to produce a set of stable software requirements**. Customers cannot predict how a system will affect their work or which operations should be automated; the real requirements often become clear only after users have tried the system.
- Software therefore has to **evolve quickly** to reflect changing business needs.
- **Plan-driven development** (specify everything, then design, build and test) is essential for some types of system, e.g. safety-critical control systems, but it **does not meet these business needs**: each change forces rework, so software arrives long after it was specified.
- **Agile development methods** (e.g. Extreme Programming, Scrum, DSDM) emerged in the **late 1990s**. Their aim was to radically reduce the delivery time for working software systems.

IN SIMPLE WORDS

Ordering a tailor-made suit. **Plan-driven**: you give every measurement on day one and collect the suit months later; if your size or taste changed, alterations are expensive. **Agile**: the tailor gives you a fitting every two weeks and adjusts as you go.

What is agile development? ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Agile methods — incremental development methods that focus on rapid software development, frequent releases of the software, reducing process overheads by minimizing documentation, and producing high-quality code. Customer representatives are involved directly in development.

An **increment** is a small, working piece of new functionality added in each version; a **stakeholder** is anyone with an interest in the system (users, customers, managers, regulators). Agile development is characterized by:

1. **Interleaved specification, design and implementation.** There is no detailed specification; design documentation is minimal or generated automatically.
2. **A series of versions (increments)**, with stakeholders involved in specifying and evaluating each version.
3. **Frequent delivery** of new versions for evaluation, typically every 2–3 weeks.
4. **Extensive tool support**, e.g. automated testing tools, configuration management and integration tools.
5. **Minimal documentation, focus on working code**; informal communication instead of formal documents.

Plan-driven vs agile development ★★★ MUST KNOW

Plan-driven Based around **separate development stages**, with the outputs of each stage **planned in advance**. Not necessarily waterfall: plan-driven *incremental* development is possible. **Iteration occurs within activities.**

Agile Specification, design, implementation and testing are **interleaved**, and the outputs are **decided through negotiation** during development. **Iteration occurs across activities.**

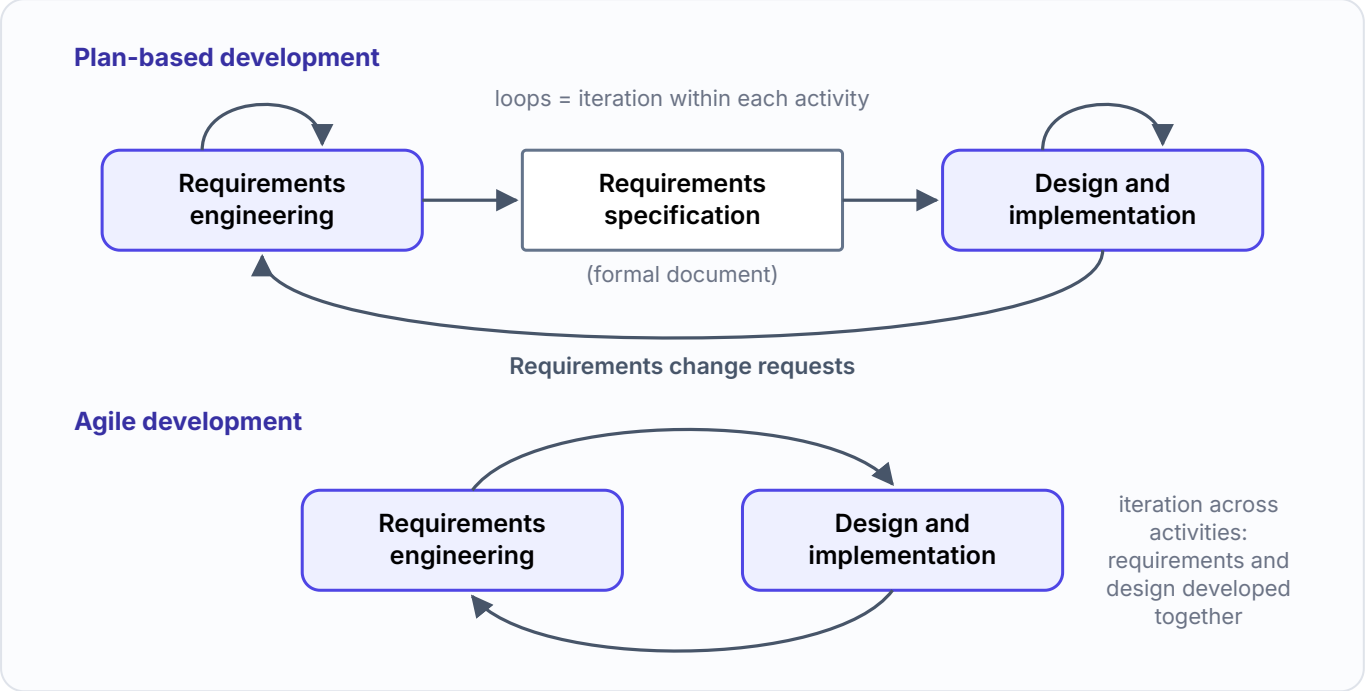


Fig 3.1 Plan-driven and agile development. Plan-based: a formal requirements specification sits between the stages, and a change request must go all the way back. Agile: requirements and design feed each other continuously. Sommerville Fig 3.1 · Your slides, slide 5

Aspect	Plan-driven development	Agile development
Basic idea	Separate stages; outputs planned in advance	Activities interleaved; outputs negotiated during development
Iteration	Within activities	Across activities
Communication	Formal documents between stages	Informal; minimal documentation, working code
Delivery	Often long after the software was specified	New versions every 2–3 weeks
Change	Causes rework and retesting of design and code	Expected; handled in the next increment
Best for	Safety-critical, large, long-life or regulated systems	Small/medium products and business systems with changing requirements

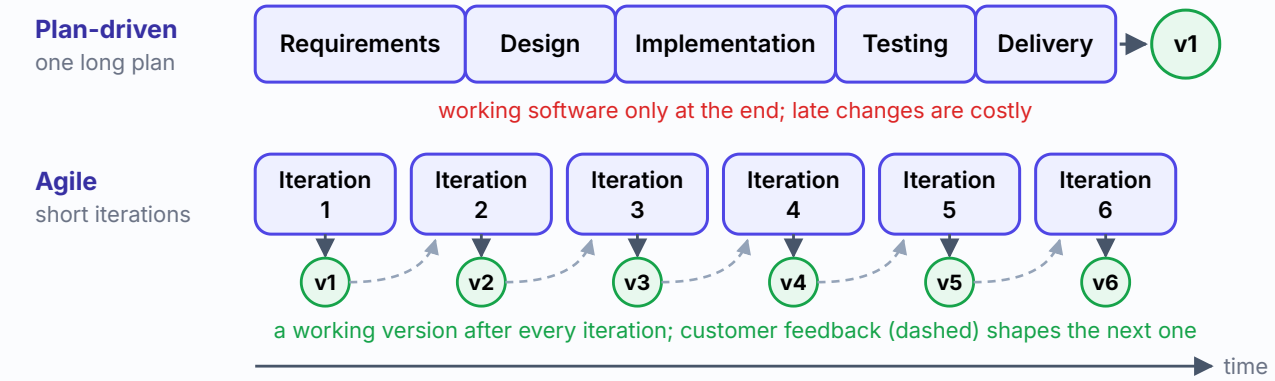


Fig 3.2 One big plan vs many short iterations. Drawn for this guide

⚠ **WATCH OUT**

Plan-driven is not the same as waterfall: plan-driven increments are possible if planned in advance. And agile is not "code only": an iteration may produce models and documentation. Real projects usually mix both (3.9).

✎ **EXAM TIP**

"Differentiate plan-driven and agile": draw Fig 3.1 + 5–6 table rows; say **"iteration within activities" vs "iteration across activities"**.

3.2 Agile methods ★★★ MUST KNOW

The problem agile methods solved ★★ IMPORTANT

In the 1980s and early 1990s, better software was thought to come from **careful project planning, formalized quality assurance, tool-supported analysis and design methods, and rigorous controlled processes**. This suited **large, long-lived systems** (aerospace, government) built by many teams. Applied to **small and medium business systems**, the overhead **dominated**: more time went on *how* to develop than on programming and testing, and every requirements change meant reworking the specification and design.

Dissatisfaction with these overheads led to agile methods, which:

- **focus on the code** rather than the design;
- are based on an **iterative approach** to software development;
- are intended to **deliver working software quickly** and **evolve it quickly** to meet changing requirements.

> ★ KEY POINT
> **Aim of agile methods:** to reduce overheads in the software process (e.g. by limiting documentation) and to be able to **respond quickly to changing requirements without excessive rework**.

The Agile Manifesto ★★★ MUST KNOW

Issued by the leading developers of agile methods (agilemanifesto.org). Learn it word for word:

> *"We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
>
> - **Individuals and interactions** over processes and tools
> - **Working software** over comprehensive documentation
> - **Customer collaboration** over contract negotiation
> - **Responding to change** over following a plan
>
> *That is, while there is value in the items on the right, we value the items on the left more."*

In simple words: people talking to each other matter more than fixed procedures or tools; progress is measured by software that runs, not by documents; work with the customer rather than argue over a contract; adapt when requirements change instead of clinging to an old plan.

> ⚠ WATCH OUT
> The manifesto does **not** call processes, documentation, contracts or plans useless; it values the left-hand items *more*. Writing "agile means no documentation" loses marks.

The principles of agile methods ★★★ MUST KNOW

All agile methods share these principles, based on the manifesto (Sommerville Fig 3.2):

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is to provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments, with the customer specifying the requirements to be included in each increment.
People, not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change, and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

MEMORY TRICK

"Can I Pass Every Module?" gives **C**ustomer involvement, **I**ncremental delivery, **P**eople not process, **E**mbrace change, **M**aintain simplicity.

Agile method applicability ★★ IMPORTANT

1. **Product development**: a software company developing a **small or medium-sized product for sale**. Virtually all software products and apps are now developed using an agile approach.
2. **Custom system development within an organization**, where there is a **clear commitment from the customer** to be involved in development and **few external rules and regulations** affect the software.

They fit because the customer can talk to the team continuously, the software is stand-alone, and a small **co-located** team can communicate informally. A safety-critical, regulated system such as the textbook's **insulin pump** needs plan-driven analysis instead.

3.3 Agile techniques: Extreme Programming ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Extreme Programming (XP) — a very influential agile method, developed in the late 1990s, that introduced a range of agile techniques. It takes an **"extreme" approach to iterative development**: new versions may be built several times per day, increments are delivered to customers every 2 weeks, and all tests must be run for every build; the build is only accepted if the tests run successfully.

The XP release cycle ★★★ MUST KNOW

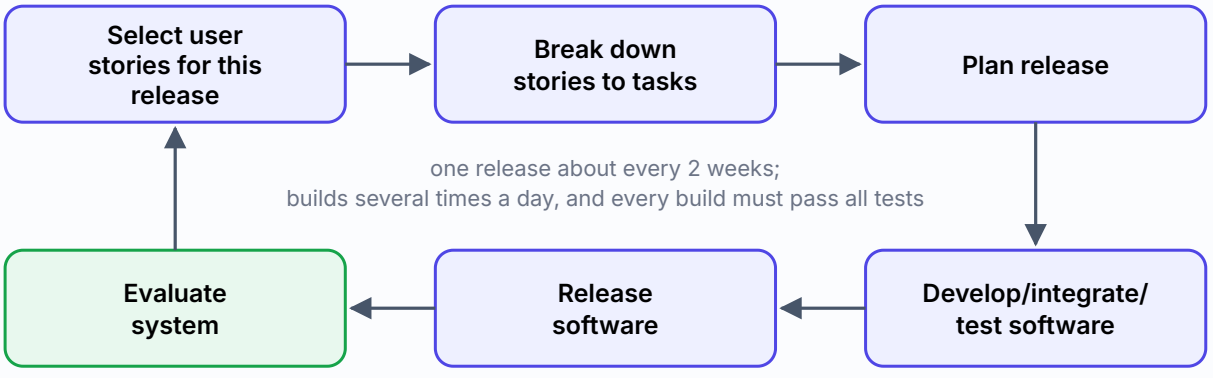


Fig 3.3 The XP release cycle: the process that produces one increment of the system. Sommerville Fig 3.3 · Your slides, slide 14

In words: (1) the customer **selects the user stories** for this release by priority and time available; (2) developers **break the stories into tasks**; (3) tasks are estimated and the **release is planned**; (4) pairs **develop, integrate and test** (tests first, all tests must pass); (5) the **software is released** to the customer; (6) the customer **evaluates the system**, and new or changed stories start the next cycle.

Extreme programming practices ★★★ MUST KNOW

All ten (Sommerville Fig 3.4). A **unit test** checks one small piece of code; **integration** means adding finished code into the whole system.

Principle or practice	Description
1. Incremental planning	Requirements are recorded on **story cards**, and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development "tasks" (see Figs 3.4 and 3.5).
2. Small releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
3. Simple design	Enough design is carried out to meet the current requirements and no more.
4. Test-first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
5. Refactoring	All developers are expected to refactor the code continuously as soon as potential code improvements are found. This keeps the code simple and maintainable.
6. Pair programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
7. Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. Anyone can change anything.
8. Continuous integration	As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
9. Sustainable pace	Large amounts of overtime are not considered acceptable, as the net effect is often to reduce code quality and medium-term productivity.
10. On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an XP process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

MEMORY TRICK

Five groups. **Plan**: incremental planning, small releases, on-site customer. **Design**: simple design, refactoring. **Code**: pair programming, collective ownership, continuous integration. **Test**: test-first. **People**: sustainable pace. (3 + 2 + 3 + 1 + 1 = 10.)

XP and agile principles ★★ IMPORTANT

Agile principle	How XP supports it
Incremental development	**Small, frequent system releases**; simple customer stories decide what goes into each increment.
Customer involvement	**Full-time customer engagement** with the team; the customer also defines acceptance tests.
People, not process	**Pair programming, collective ownership** and a process that **avoids long working hours**.
Embrace change	**Regular system releases**, plus test-first development, refactoring and continuous integration.
Maintain simplicity	**Constant refactoring** of code and simple designs that do not anticipate future changes.

Influential XP practices ★★ IMPORTANT

XP has a **technical focus** and is **not easy to integrate with management practice** in most organizations. So although agile development uses XP practices, **the method as originally defined is not widely used**; companies pick the practices they want, often alongside Scrum (3.8). The **key practices** are: **user stories for specification, refactoring, test-first development and pair programming** (sections 3.4–3.7).

3.4 User stories for requirements ★★★ MUST KNOW

Requirements always change, so agile methods have no separate requirements phase; they **combine requirements elicitation** (finding out what users need) **with development**.

> **DEFINITION — LEARN THIS LINE**
>
> **User story** — a scenario of use that might be experienced by a system user, written briefly on a **story card** by the customer and the team. The team aims to implement it in a future release.

- In XP, a **customer or user is part of the XP team** and is responsible for making decisions on requirements.
- User requirements are expressed as **user stories or scenarios**.
- These are written on **cards**, and the development team breaks them down into **implementation tasks**. These tasks are the basis of **schedule and cost estimates**.
- The **customer chooses the stories** for the next release based on their priorities and the schedule estimates (about two weeks of work). Unimplemented stories may change or be discarded as requirements change.

Example: the "prescribing medication" story ★★★ MUST KNOW

From **Mentcare**, the textbook's patient information system for mental health care. A **formulary** is the approved list of drugs and their recommended doses. (The textbook tells the story about Kate, a doctor; the slides use "you".)

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either 'current medication', 'new medication' or 'formulary'.

If you select 'current medication', you will be asked to check the dose. If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose 'new medication', the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

If you choose 'formulary', you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click 'OK' or 'Change'. If you click 'OK', your prescription will be recorded on the audit database. If you click 'Change', you reenter the 'Prescribing medication' process.

Fig 3.4 A "prescribing medication" story card. Sommerville Fig 3.5 · Your slides, slide 20

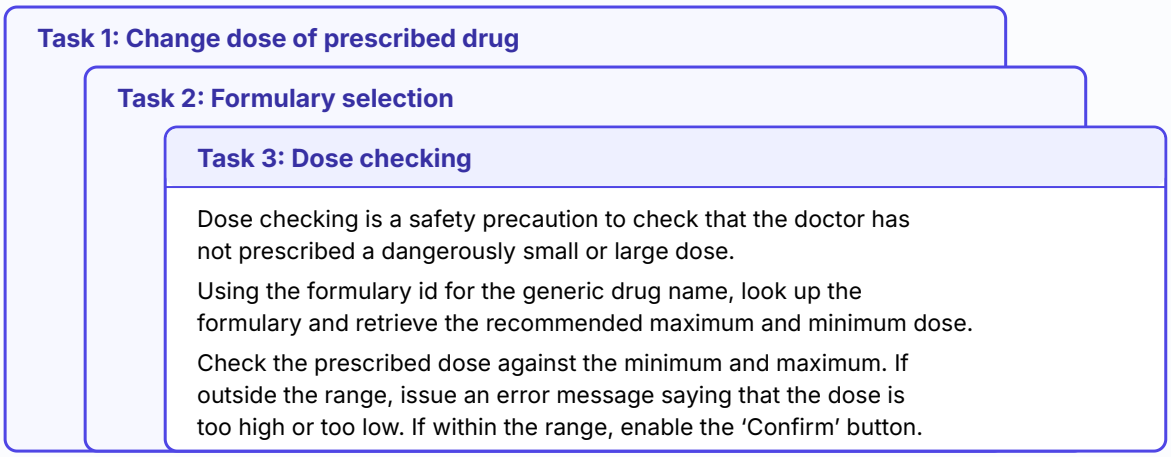


Fig 3.5 Examples of task cards for prescribing medication (Task 3 shown in full). Sommerville Fig 3.6 · Your slides, slide 21

STRENGTHS

- ✓ People relate to stories much more easily than to a requirements document or use cases.
- ✓ They get users involved in suggesting requirements.
- ✓ Tasks give a direct basis for estimates and planning.

PROBLEM: COMPLETENESS

- ✗ Hard to judge if enough stories cover all essential requirements.
- ✗ Hard to judge if one story gives a true picture of an activity.
- ✗ Experienced users leave things out when describing their work.

3.5 Refactoring ★★★ MUST KNOW

DEFINITION — LEARN THIS LINE

Refactoring — constant code improvement: the programming team look for possible improvements to the software and implement them immediately, even where there is no immediate need for them. In plain words, tidying up code so it is clearer and better structured, without adding features.

	Conventional wisdom: design for change	XP: refactoring
Idea	Spend time and effort anticipating changes, as this reduces costs later in the life cycle	Not worthwhile: **changes cannot be reliably anticipated** (expected ones often never come)
Practice	Add generality to the design up front	**Constant code improvement**, so changes are easy when they have to be implemented

Why refactor? ★★★ MUST KNOW

In incremental development, **local changes degrade the software structure** (workarounds, duplicated code), making each further change harder. Refactoring prevents this:

- The team looks for possible software improvements and makes them **even where there is no immediate need**.
- It improves the **understandability** of the software and so **reduces the need for documentation**.
- **Changes are easier** to make because the code is well-structured and clear.

- **However**, some changes require **architecture refactoring**, and this is **much more expensive**. Delivery pressure can also delay refactoring.

Examples of refactoring ★★★ MUST KNOW

1. **Re-organization of a class hierarchy** to remove duplicate code (Fig 3.6).
2. **Tidying up and renaming attributes and methods** to make them easier to understand.
3. **Replacement of inline code** with calls to methods that have been included in a program library.

Development environments usually provide **refactoring tools** that find dependencies between code sections and make global changes.

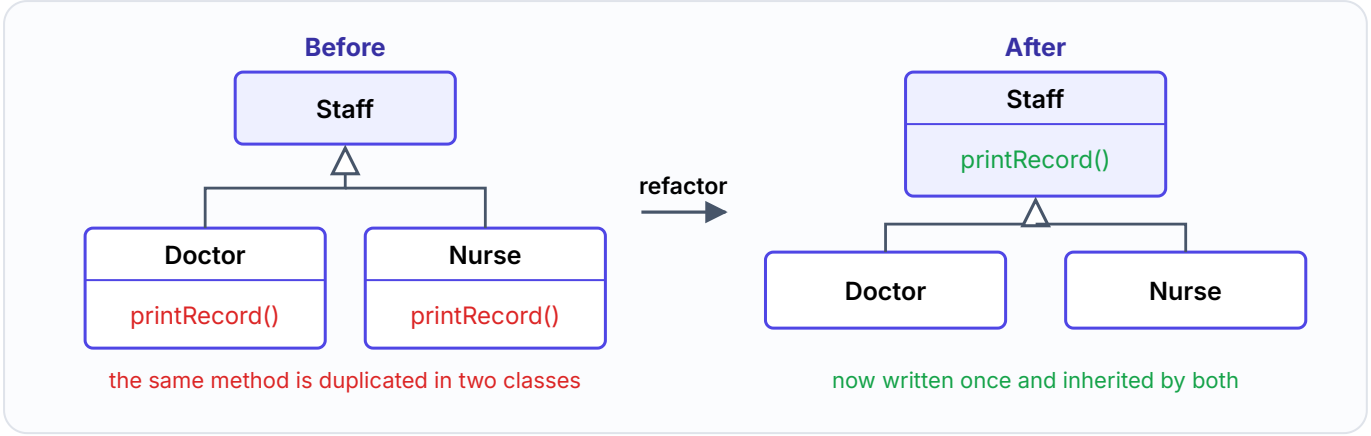


Fig 3.6 Refactoring a class hierarchy to remove duplicate code. Behaviour is unchanged, but *printRecord()* is now changed in one place. Drawn for this guide

3.6 Test-first development ★★★ MUST KNOW

Incremental development has no full specification for an external test team to work from. So in XP, **testing is central**: the program is **tested after every change**, and development cannot proceed until all tests pass. **XP testing features:**

1. **Test-first development**: tests are written before the code.
2. **Incremental test development from scenarios** (user stories and their tasks).
3. **User involvement** in test development and validation.
4. **Automated test harnesses** run all component tests each time a new release is built.

Test-driven development (TDD) ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Test-first (test-driven) development** — instead of writing code and then tests for it, you write the tests **before** the code, so they can be run while the code is written and problems are found during development.

- **Writing tests before code clarifies the requirements** to be implemented. A test defines the interface and the expected behaviour, so ambiguities must be cleared up before coding.
- **Tests are written as programs rather than data** so that they can be executed automatically. The test includes a check that it has executed correctly. It usually relies on a testing framework such as **JUnit** (for Java).

- **All previous and new tests are run automatically** when new functionality is added, checking that it has not introduced errors.
- Each task card generates one or more **unit tests**, and testing never falls behind coding ("test-lag" is avoided).

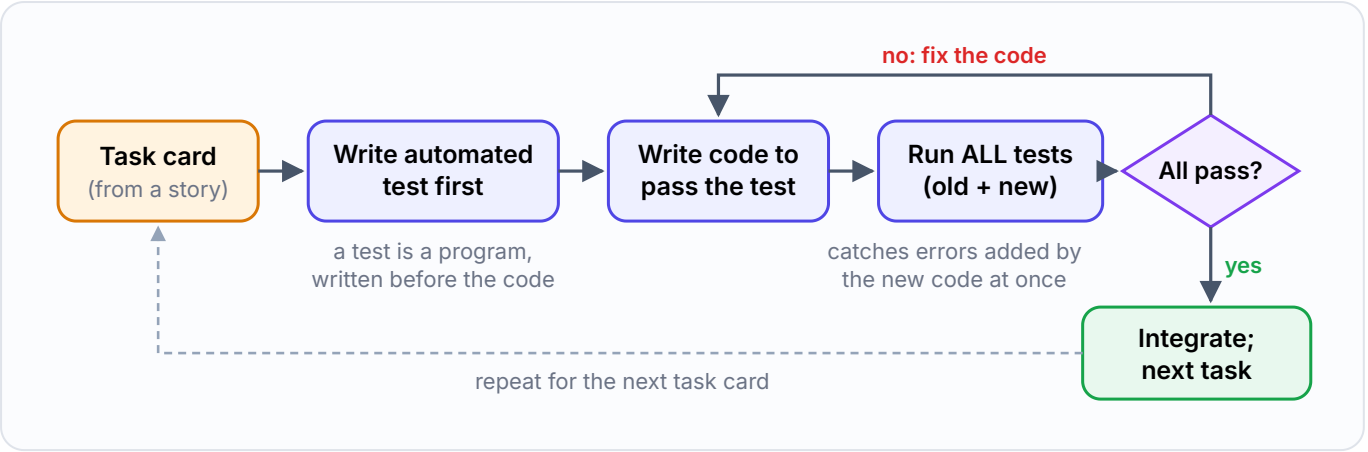


Fig 3.7 The test-first rhythm for one task. Drawn for this guide

Example: test case for dose checking ★★ IMPORTANT

This test checks Task 3 of Fig 3.5. It covers the wrong cases as well as the correct one, and would be written as a JUnit program *before* the dose-checking code.

Test 4: Dose checking

Input:
1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Output:
OK or error message indicating that the dose is outside the safe range.

Tests:
1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Fig 3.8 Test case description for dose checking. Sommerville Fig 3.7 · Your slides, slide 28

Customer involvement

- The customer helps develop **acceptance tests** (tests with customer data that check real needs are met) for the stories in the next release.
- The customer in the team **writes tests as development proceeds**, so all new code is validated against what the customer needs.
- **But** customers have limited time, cannot work full-time with the team, and may feel that giving the requirements was enough, so they may be reluctant to help with testing.

Test automation

- Tests are written as **executable components before the task is implemented**.
- They should be **stand-alone**, **simulate the submission of input**, and **check the result meets the output specification**.
- An **automated test framework** (e.g. JUnit) makes it easy to write executable tests and submit a set of tests for execution.
- There is **always a set of tests that can be quickly run**, so problems introduced by new code are caught immediately.

Problems with test-first development ★★★ MUST KNOW

1. **Programmers prefer programming to testing** and sometimes take short cuts, e.g. incomplete tests that do not check for all possible exceptions.
2. **Some tests are very difficult to write incrementally**, e.g. unit tests for the "display logic" and workflow between screens of a complex user interface.
3. **It is difficult to judge the completeness of a set of tests.** You may have many tests but not complete coverage, so crucial parts stay untested. A big test set can give a false impression that the system is correct.

3.7 Pair programming ★★★ MUST KNOW

> DEFINITION — LEARN THIS LINE
>
> **Pair programming** — programmers work in pairs, sitting together at the same computer, developing the code together. Pairs are created dynamically, so that all team members work with each other during development.

Advantages

1. **Common ownership of code; knowledge spread across the team.** The team as a whole owns the software and shares responsibility for its problems.
2. **Informal review process**: each line of code is looked at by more than one person. This finds fewer errors than formal inspections but is cheaper and easier to organize.
3. **Encourages refactoring**, because the whole team benefits from improving the system code.
4. **Reduces project risk**: the knowledge sharing is very important because it reduces the overall risk to a project when team members leave.

Is it inefficient? ★★ IMPORTANT

Pair programming is **not necessarily inefficient**: there is some evidence that a pair is more efficient than two programmers working separately. Results are mixed, though. With student volunteers (Williams et al., 2000), productivity was **comparable** to two individuals (fewer false starts and less rework); with experienced programmers (Arisholm et al., 2007), it showed a **significant loss**. The knowledge sharing alone may make it worthwhile.

3.8 Agile project management and Scrum ★★★ MUST KNOW

Why agile project management? ★★ IMPORTANT

The principal responsibility of software project managers is to manage the project so that the software is **delivered on time and within the planned budget**. The standard approach is **plan-driven**: managers draw up a plan showing **what** should be delivered, **when**, and **who** will work on the deliverables. Agile project management requires a **different approach**, adapted to incremental development and agile practices, which still gives managers **visibility** of progress.

Scrum ★★★ MUST KNOW

> **DEFINITION — LEARN THIS LINE**
>
> **Scrum** — an agile method that focuses on **managing iterative development** rather than on specific agile practices. It provides a project management framework centred round a set of **sprints**, which are fixed time periods when a system increment is developed.

Scrum uses new terms (e.g. *ScrumMaster*, not *project manager*) and does not mandate practices such as pair programming, so it fits existing company practice; it is the **most widely used** agile method. Your slides give **three phases**:

1. **Outline planning phase**: establish the general objectives for the project and design the software architecture.
2. **A series of sprint cycles**, where each cycle develops an increment of the system.
3. **Project closure phase**: wraps up the project, completes required documentation such as system help frames and user manuals, and assesses the lessons learned.

Scrum terminology ★★★ MUST KNOW

Scrum term	Definition (Sommerville Fig 3.8)
Development team	A **self-organizing** group of software developers, which should be **no more than 7 people**. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. "Potentially shippable" means that it is in a **finished state** and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	A list of **"to do" items** which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to **identify product features or requirements, prioritize these** for development and **continuously review the product backlog** to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.
Scrum	A **daily meeting** of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a **short face-to-face meeting** that includes the whole team (the daily Scrum or stand-up).
ScrumMaster	Responsible for **ensuring that the Scrum process is followed** and guides the team in the effective use of Scrum. He or she is responsible for **interfacing with the rest of the company** and for ensuring that the Scrum team is **not diverted by outside interference**. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A **development iteration**. Sprints are usually **2–4 weeks** long.
Velocity	An **estimate of how much product backlog effort that a team can cover in a single sprint**. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Also used: the **sprint backlog** (the items selected for one sprint) and the **Scrum board** (a shared whiteboard with post-it notes showing the sprint backlog, work done, staff unavailability, etc.).

The Scrum sprint cycle ★★★ MUST KNOW

1. **Product backlog** (Fig 3.9): the starting point for planning; the list of work to be done on the project (features, requirements, user stories, engineering tasks).
2. **Review work and select items**: the Product Owner prioritizes the backlog. In the **selection phase** the **whole project team works with the customer** to select the features and functionality to be developed during the sprint.
3. **Plan sprint**: the team estimates the work using its **velocity**, producing the **sprint backlog**. Once these are agreed, the **team organizes itself** to develop the software.
4. **Sprint**: fixed length, normally **2–4 weeks**. The **team is isolated from the customer and the organization**; all communication goes through the **Scrum master**, whose role is to **protect the team from external distractions**. Daily Scrums track progress.
5. **Review sprint**: at the end of the sprint the work done is **reviewed and presented to stakeholders**. The **next sprint cycle then begins**.

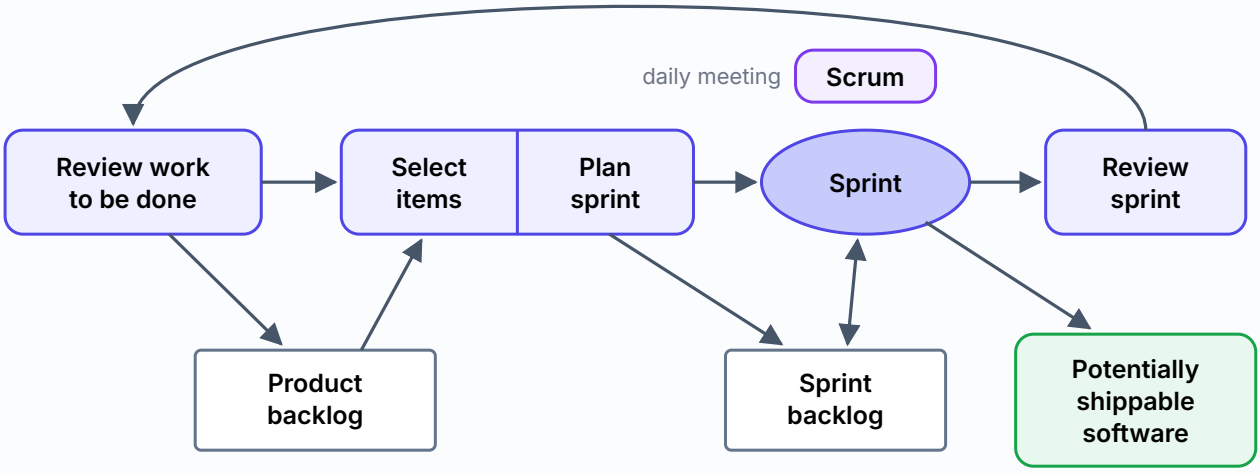


Fig 3.9 The Scrum sprint cycle. Input: the product backlog; output of each cycle: potentially shippable software. Sommerville Fig 3.9 · Your slides, slide 38

⚠ WATCH OUT

Sprints are **never extended** to finish late work; unfinished items go back to the product backlog.



Fig 3.10 One sprint as a timeline. Drawn for this guide

Teamwork in Scrum ★★ IMPORTANT

- The **Scrum master is a facilitator** who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside the team.
- The **whole team attends short daily meetings (Scrums)** to share progress since the last meeting, problems that have arisen and plans for the following day, so everyone knows what is going on and can **re-plan short-term work** to cope with problems.

Scrum benefits ★★★ MUST KNOW

1. The product is broken down into a set of **manageable and understandable chunks**.
2. **Unstable requirements do not hold up progress.**
3. The **whole team has visibility of everything**, so team communication is improved.
4. Customers see **on-time delivery of increments** and gain **feedback** on how the product works.
5. **Trust between customers and developers** is established, and a positive culture is created in which everyone expects the project to succeed.

MEMORY TRICK

Benefits = **C-U-V-F-T**: **C**hunks, **U**nstable requirements no problem, **V**isibility, **F**eedback and on-time delivery, **T**rust ("Clever Users Value Fast Teams").

Distributed Scrum ★★ IMPORTANT

Scrum was designed for **co-located** teams; today teams are often spread worldwide (cheaper staff, specialist skills, 24-hour working). Distributed Scrum needs:

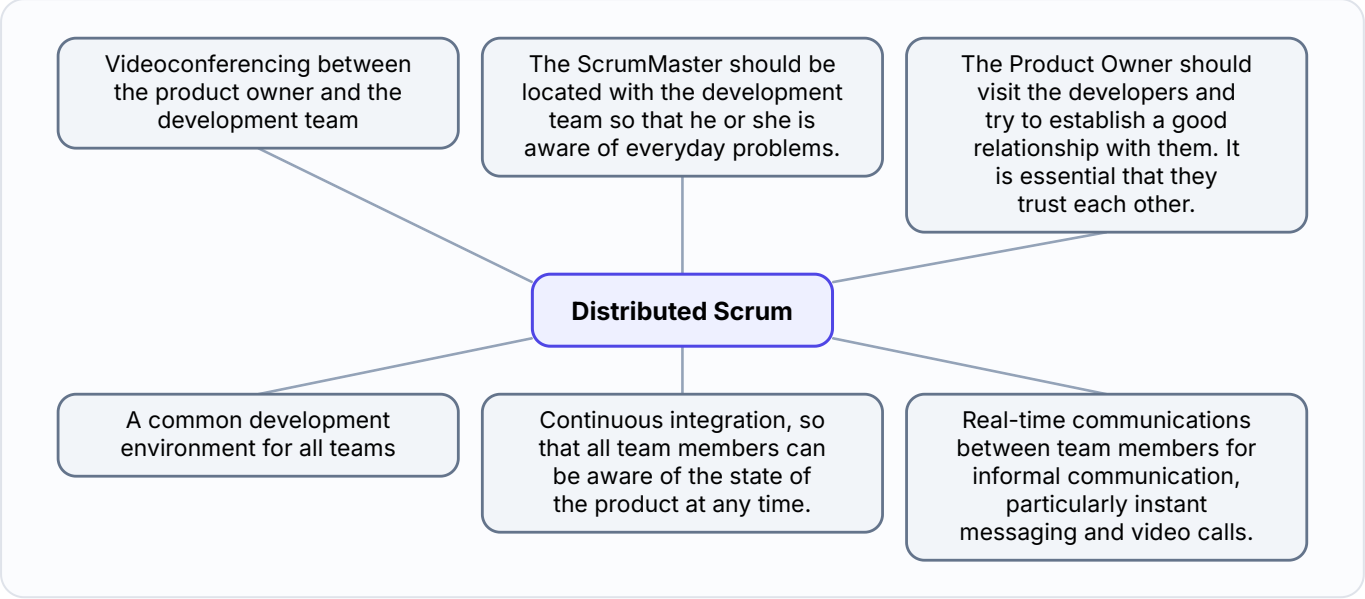


Fig 3.11 Distributed Scrum. Sommerville Fig 3.10 · Your slides, slide 43

3.9 Scaling agile & agile vs plan-driven in practice ★ READ ONCE

This section covers slides 44–64, which are in your slides but not named in the syllabus list. Read it once; learn only "Agile vs plan-driven: which factors decide" (★★).

Scaling agile methods ★ READ ONCE

Agile methods succeed for **small and medium-sized projects** built by a **small co-located team** (thanks, it is argued, to better communication). Scaling means coping with **larger, longer projects with multiple teams**, perhaps in different places. **Scaling up**: agile for **large systems** too big for a small team. **Scaling out**: agile

across a large organization with long development experience. Keep the **fundamentals**: flexible planning, frequent releases, continuous integration, test-driven development, good communications.

Practical problems with agile methods ★ READ ONCE

Problem	Detail
Contracts	Agile informality is **incompatible with the legal approach to contracts**. Specification-based contracts preclude interleaving, so contracts must **pay for developer time, not functionality**, which legal departments see as high risk.
Maintenance	Agile suits new development, yet most money goes on **maintenance**. Issues: are agile systems **maintainable** with minimal documentation, and can agile **evolve a system** for change requests? Key problems: **lack of product documentation; keeping customers involved; development team continuity**. Agile relies on the team knowing what has to be done, but in long-lifetime systems the original developers will not always be there.
Distributed teams	Agile methods are designed for **small co-located teams**, yet much development now uses worldwide distributed teams.

Agile principles and organizational practice ★ READ ONCE

Principle	Why it is hard in practice
Customer involvement	Needs a customer willing and able to spend time with the team and represent all stakeholders; representatives are busy, and external stakeholders (regulators) are hard to represent.
Embrace change	Prioritizing changes is extremely hard when many stakeholders differ.
Incremental delivery	Short-term planning doesn't fit longer business and marketing cycles; marketing may need features months ahead.
Maintain simplicity	Under delivery pressure there may be no time for desirable simplifications.
People, not process	Some people's personalities don't suit the intense involvement of agile methods.

Agile vs plan-driven: which factors decide ★★ IMPORTANT

Most projects include **elements of both** plan-driven and agile processes. The right balance depends on three groups of factors:

Issue	Questions to ask (plan-driven is favoured when…)
System issues	**How large?** Too large for a small co-located team that communicates informally. **What type?** Needs a lot of analysis and a detailed design before implementation. **Lifetime?** Long-lifetime, so documentation must tell the support team the developers' intentions. **External regulation?** Regulated, so detailed documentation is needed for the safety case.
People and teams	**How skilled are the designers and programmers?** Agile may need higher skill than translating a detailed design into code. **How is the team organized?** Distributed teams may need design documents. **What support technology?** Without design documents, IDE visualization and program-analysis tools are essential.
Organizational issues	Traditional engineering organizations have a **plan-based culture**. **Is a detailed specification standard practice? Will customer representatives be available to give feedback on increments** (is incremental delivery realistic)? **Can informal agile fit a culture of detailed documentation?**

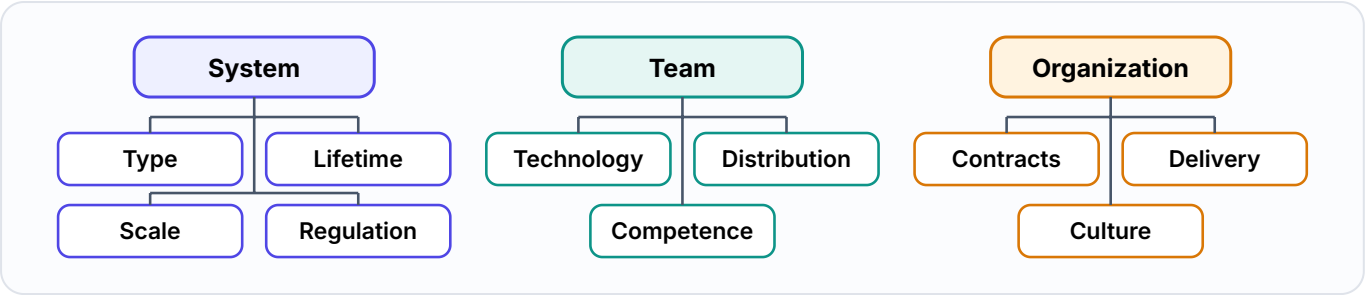


Fig 3.12 Factors influencing the choice of plan-based or agile development. Sommerville Fig 3.12 · Your slides, slide 54

Agile methods for large systems ★ READ ONCE

Six factors make large systems hard for agile (slide 60 draws them around "Large software system"):

- **System of systems**: separate communicating systems, each built by its own team, often in different places and time zones.
- **Brownfield development**: interaction with existing systems resists incremental development.
- **System configuration**: much of the work is configuration, not original code.
- **Regulatory constraints**: external rules limit how they are developed.
- **Prolonged procurement**: long timescales make coherent teams hard to keep.
- **Diverse stakeholders**: impossible to involve them all.

IBM's agility at scale model (slide 61; three nested circles). **Core agile development**: value-driven life-cycle, self-organizing teams, focus on construction. **Disciplined agile delivery**: risk+value driven life-cycle, self-organizing with appropriate governance framework, full delivery life-cycle. **Agility at scale**: disciplined agile delivery where scaling factors apply: large team size, geographic distribution, regulatory compliance, domain complexity, organization distribution, technical complexity, organizational complexity, enterprise discipline.

Scaling up to large systems

1. A **completely incremental approach to requirements engineering is impossible**.
2. There **cannot be a single product owner** or customer representative.
3. It is **not possible to focus only on the code** of the system.
4. **Cross-team communication mechanisms** have to be designed and used.
5. **Continuous integration is practically impossible**; keep frequent builds and regular releases.

Multi-team Scrum

1. **Role replication**: each team has a Product Owner for its work component, and a ScrumMaster.
2. **Product architects**: one per team; together they design and evolve the overall architecture.
3. **Release alignment**: release dates aligned so a demonstrable, complete system is produced.
4. **Scrum of Scrums**: a daily meeting of team representatives to discuss progress and plan work.

Agile across organizations is hard because of (1) **project managers** reluctant to risk a new approach; (2) bureaucratic **quality procedures and standards**; (3) a wide **range of skill levels** (agile works best with high skills); (4) **cultural resistance** after long use of conventional processes.

Chapter 3 in one page

Core definitions

- **Agile methods**: incremental; rapid development, frequent releases, minimal documentation, high-quality code, customer involved.
- **Plan-driven**: stages, outputs planned in advance, iteration *within* activities. **Agile**: interleaved, outputs negotiated, iteration *across* activities.
- **User story**: a scenario of use on a story card, broken into tasks. **Refactoring**: constant code improvement. **Test-first**: automated tests written before the code.

Agile Manifesto (left over right)

- Individuals and interactions / processes and tools · Working software / comprehensive documentation · Customer collaboration / contract negotiation · Responding to change / following a plan

5 principles ("Can I Pass Every Module?")

- Customer involvement · Incremental delivery · People, not process · Embrace change · Maintain simplicity

10 XP practices

- Incremental planning · Small releases · Simple design · Test-first · Refactoring · Pair programming · Collective ownership · Continuous integration · Sustainable pace · On-site customer

XP release cycle

- Select stories → Break into tasks → Plan release → Develop/integrate/test → Release → Evaluate → repeat

Problems to remember

- Stories: completeness. Test-first: short-cut tests, hard UI tests, unknown coverage. Pairing: mixed productivity evidence.

Scrum

- Product Owner · ScrumMaster · Development team (≤ 7) · product backlog · sprint (2–4 weeks) · velocity · daily Scrum · potentially shippable increment.
- Cycle: backlog → select items → plan sprint → sprint → review sprint → repeat. Benefits: C-U-V-F-T.

Key points

- Many real methods mix plan-based and agile. Scaling agile to large systems is hard: they need up-front design and some documentation.

Diagrams to practise

- Fig 3.1 · Fig 3.3 · Fig 3.5 · Fig 3.9 · Fig 3.11

Likely exam questions

2M **Define agile methods. What is their aim?**

How to answer → Definition (3.1) + aim: reduce overheads, respond quickly to change without excessive rework (3.2).

4M **State the Agile Manifesto.**

How to answer → Opening sentence, four "X over Y" lines verbatim, closing sentence.

6M **Explain the principles of agile methods.**

How to answer → Five-row table in 3.2.

6M **Differentiate plan-driven and agile development with a neat diagram.**

How to answer → Fig 3.1 → comparison table (iteration within vs across activities, documents, delivery, change, best use).

10M **Explain Extreme Programming: its release cycle and practices.**

How to answer → Definition → Fig 3.3, one line per box → all 10 practices.

5M **How does XP reflect the principles of agile methods?**

How to answer → Five-row table "XP and agile principles" (3.3).

8M **What are user stories? Explain with the "prescribing medication" example.**

How to answer → Definition → 4 points → story (Fig 3.4) → task cards (Fig 3.5) → completeness problem.

5M **What is refactoring? Why is it needed? Give examples.**

How to answer → Definition; design for change vs refactoring; benefits and architecture caveat; 3 examples (Fig 3.6).

8M **Explain test-first development in XP and its problems.**

How to answer → 4 testing features → TDD points (JUnit) → Fig 3.8 → customer role, automation → 3 problems.

5M **What is pair programming? State its advantages.**

How to answer → Definition → 4 advantages → productivity evidence.

2M **Define: sprint, velocity, product backlog, ScrumMaster, Product Owner.**

How to answer → Scrum terminology table (3.8).

10M **Explain the Scrum sprint cycle with a neat diagram.**

How to answer → Fig 3.9 → 5 steps → never extended → ScrumMaster's role → three phases of Scrum.

5M **Explain teamwork in Scrum and the benefits of Scrum.**

How to answer → ScrumMaster duties + daily Scrum → 5 benefits.

4M **What are the requirements of Distributed Scrum?**

How to answer → Fig 3.11, six points.

6M **Which factors decide between agile and plan-driven development?**

How to answer → Fig 3.12 → system, team and organizational questions (3.9).

EXAM DAY

Exam-Day Playbook

How to turn what you studied into marks: answer structure, command words, diagrams and the comparison questions to expect.

Writing answers that score

2 marks

- One-line **definition** in textbook words
- Plus **one** example or key feature
- 2–3 lines total. Don't write an essay.

4–6 marks

- Definition
- A small diagram or a table, if one exists
- 4–6 points, each with a bold keyword and one explaining line
- One example

8–10 marks

- Definition / introduction (2 lines)
- **Neat labelled diagram**, drawn first
- Explain each box or phase under its own sub-heading
- Advantages and disadvantages, or a comparison
- When to use it + a case-study example
- One-line conclusion

★ KEY POINT

Examiners mark **keywords**, **structure** and **diagrams**. Underline the keywords (the highlighted phrases in this guide), number your points, and leave space between them.

Diagram rules

1. **Draw first, write after.** The diagram tells you what to explain, and it scores even if time runs out.
2. **Label every box and arrow** using the textbook's names (for example "Requirements definition", not "Req").
3. **Give it a caption**: *Fig: The waterfall model.*
4. **Use boxes for activities, arrows for flow**, and dashed arrows for feedback or iteration.
5. **Keep it big and clean.** Half a page is fine for a 10-mark question.

Command words decoded

If the question says…	…it wants
Define / What is	The exact definition, plus one example. Short.
List / Enumerate	The complete list, numbered. One line each is enough.

If the question says…	…it wants
Explain / Describe	Definition + points with explanation + diagram if one exists + example.
With a neat diagram	The diagram carries a large share of the marks. Draw it carefully and explain every part.
Differentiate / Compare / Distinguish	A **table** with a "Basis" column and 5–7 rows. Add one line each of "use A when…, use B when…".
Discuss / Justify	Points for and against, then your conclusion with a reason.
Apply to / For the given case study	Name the system in every point. Generic answers lose marks here.

✎ **CASE-STUDY QUESTIONS**

When a question names a system (insulin pump, Mentcare, weather station, iLearn, or something new like a library or ATM system), connect every point to it: "*Because the insulin pump is safety-critical, …*". If the system is new, pick the textbook system it most resembles and reuse that reasoning.

Diagrams you must be able to draw

Every textbook or slide diagram in this guide. Tick each one when you can draw it from memory with all labels. Diagrams drawn only to help understanding are not listed.

Ch 1 · Introduction

☐ **Fig 1.3** The four essential attributes of good software

☐ **Fig 1.4** Software engineering is one part of system engineering and builds on computer science

☐ **Fig 1.5** Insulin pump hardware architecture

☐ **Fig 1.6** Activity model of the insulin pump (UML)

☐ **Fig 1.7** Organisation of the Mentcare system

☐ **Fig 1.8** The weather station's environment (UML packages with the «system» stereotype)

☐ **Fig 1.9** The iLearn architecture

Ch 2 · Software Processes

☐ **Fig 2.1** The waterfall model

☐ **Fig 2.2** Incremental development

☐ **Fig 2.3** Reuse-oriented software engineering

☐ **Fig 2.4** The V-model as drawn by Pressman

☐ **Fig 2.5** The prototyping paradigm

☐ **Fig 2.6** A typical spiral model

☐ **Fig 2.7** The requirements engineering process

☐ **Fig 2.8** A general model of the design process

☐ **Fig 2.9** Stages of testing (slide 28 labels the last stage "Acceptance testing")

☐ **Fig 2.10** Testing phases in a plan-driven software process (V-model)

☐ **Fig 2.11** System evolution

☐ **Fig 2.12** The process of prototype development

Ch 4 · Requirements Engineering

☐ **Fig 4.1** User and system requirements

☐ **Fig 4.2** Readers of different types of requirements specification

☐ **Fig 4.3** The eight stakeholder groups of the Mentcare system (the textbook also includes…

☐ **Fig 4.4** One non-functional requirement can generate new functional requirements and restrict…

☐ **Fig 4.5** Types of non-functional requirement

☐ **Fig 4.7** A spiral view of the requirements engineering process

☐ **Fig 4.8** The requirements elicitation and analysis process (step 4 is called "requirements…

☐ **Fig 4.9** Ethnography and prototyping for requirements analysis

☐ **Fig 4.10** Use cases for the Mentcare system

☐ **Fig 4.11** Users of a requirements document and how they use it

☐ **Fig 4.12** Requirements evolution

☐ **Fig 4.13** Requirements change management

☐ **Fig 4.14** What the insulin pump software does, with the textbook requirements that govern each step

Ch 5 · Design Concepts

☐ **Fig 5.1** Design is the last action of modeling

☐ **Fig 5.2** Translating the requirements model into the design model

☐ **Fig 2.13** Incremental delivery

☐ **Fig 2.14** The process improvement cycle

☐ **Fig 2.15** Capability maturity levels as a staircase (box names from slide 54; grey meanings from...

Ch 3 · Agile

☐ **Fig 3.1** Plan-driven and agile development

☐ **Fig 3.3** The XP release cycle

☐ **Fig 3.4** A "prescribing medication" story card

☐ **Fig 3.5** Examples of task cards for prescribing medication (Task 3 shown in full)

☐ **Fig 3.8** Test case description for dose checking

☐ **Fig 3.9** The Scrum sprint cycle

☐ **Fig 3.11** Distributed Scrum

☐ **Fig 3.12** Factors influencing the choice of plan-based or agile development

☐ **Fig 5.3** The textbook's door example

☐ **Fig 5.4** Modularity and software cost

☐ **Fig 5.8** A crosscutting concern (user validation) as scattered/tangled fragments (left) versus...

☐ **Fig 5.9** The textbook's refactoring case

☐ **Fig 5.10** Core OO ideas

☐ **Fig 5.11** The analysis class FloorPlan refined into a design class

☐ **Fig 5.13** Dimensions of the design model

☐ **Fig 5.14** Interface representation for ControlPanel

☐ **Fig 5.15** (a) UML component diagram

Ch 6 · Architectural Design

☐ **Fig 6.2** The architecture of a packing robot control system

☐ **Fig 6.4** Architectural design decisions

☐ **Fig 6.5** The five architectural strategies at a glance, and the textbook's example of a conflict

☐ **Fig 6.6** Krutchen's 4+1 view model